

TSS-PV: Traceable Secret Sharing With Public Verifiability

Abstract

High-value custodial systems require both Public Verifiability to audit key distribution and Traceability to identify insider leakage via black-box *reconstruction boxes*. Existing schemes achieve one property but not both, leaving practical systems exposed to either undetectable dealer misbehavior or untraceable share leakage. Combining these properties introduces the Provenance Paradox: a verifiability-aware reconstruction box with access to verification predicates and public transcripts can reject dummy shares used for tracing because they have no provenance in the public transcript.

We present TSS-PV, the first publicly verifiable traceable secret sharing scheme that resolves this paradox. Our key insight is to inject indistinguishable dummy shares during the sharing phase itself, ensuring they are committed to the public transcript before any reconstruction box is constructed. We formalize syntax and security under a modular adversarial model: public verifiability holds against fully malicious dealers and parties; traceability identifies leaking parties after honest distribution; and non-imputability prevents a malicious dealer from framing honest parties. Both tracing properties assume a verifiability-aware perfect reconstruction box.

We instantiate TSS-PV over cyclic groups using Schnorr-based NIZKs including a new Σ -protocol for power-chain relations and a recent generic tracing framework (CRYPTO'24). Public verification costs scale linearly in the number of parties; tracing costs are quadratic. A Curve25519 prototype on commodity hardware demonstrates practicality: for 32 - 256 parties, distribution verification completes in ≈ 14 - 107 ms, tracing in ≈ 0.23 - 78 s, and trace verification in ≈ 0.13 - 26 s.

1 Introduction

The security of high-value digital assets, from blockchain bridges to root certificate authorities, relies fundamentally on (t, n) -threshold schemes. By distributing trust across n independent custodians, protocols like Shamir Secret Sharing

(SSS) [35] or Blakley Secret Sharing (BSS) [7] ensure that key reconstruction requires at least t parties, while any subset of fewer than t parties learns nothing. In adversarial deployments two practical threats persist: (i) *Dishonest Behavior*: A malicious dealer distributes malformed or inconsistent shares, causing reconstruction failures, or a custodian submits invalid shares to derail recovery. (ii) *Share Leakage*: A sub-threshold coalition ($f < t$) embeds valid shares into a reconstruction box [10], a black-box device that outputs the secret when fed $t - f$ additional valid shares, hiding leaker identities.

The Verification-Traceability Gap. Verifiable Secret Sharing (VSS) [19, 32] and its publicly verifiable extension (PVSS) [12, 24, 37] mitigate threat (i) by binding shares to dealer commitments via public transcripts, enabling verification of share distribution and submission. Orthogonally, Traceable Secret Sharing (TSS) [10, 28] addresses threat (ii) by enabling *traceability*, the ability to identify leakers by querying a reconstruction box built from any coalition of size $0 < f < t$, and *non-imputability*, the ability to protect honest parties from being falsely accused.

Boneh et al. [10] refined TSS with SSS/BSS-specific TSS and a Generic TSS (GTSS): SSS/BSS-specific schemes require only an ϵ -good box with no tracer trust, whereas GTSS applies to any symmetric scheme at the cost of requiring a perfect box and a fully trusted tracer. An ϵ -good box outputs the correct secret with probability at least ϵ over random valid query sets while the perfect box ($\epsilon = 1$) always outputs the correct secret on any $t - f$ valid shares disjoint from its embedded set, and $\perp$ otherwise. In both mechanisms, the tracer queries R with *dummy shares*, shares generated solely for tracing that are either malformed inputs (in SSS/BSS-specific schemes) or valid shares (in GTSS). TSS-PV requires a further strengthening: a *verifiability-aware* perfect box additionally has black-box access to the public transcripts and all public verification predicates, enabling it to distinguish and reject tracer queries that carry unrecognized dummy shares.

Technical Challenges. Realizing TSS-PV requires resolving the following challenges arising from the public transcripts and verification predicates.

Challenge 1: The Provenance Paradox. How can a tracer inject dummy shares when every valid share is publicly committed? Colluders can build a verifiability-aware box that selectively responds to buyers while ignoring tracer queries, since tracing queries carry dummy shares detectable via public verification predicates. The SSS/BSS-specific schemes [10] and malformed-input-based techniques [18, 27, 28] fail immediately: their inputs cannot pass verification predicates. The only remedy is a distribution mechanism where dummy shares are indistinguishable from real shares even under public verification, without compromising verifiability or the effective threshold, a property possessed only by GTSS [10], which uses valid shares as tracing inputs.

Challenge 2: The Privileged Tracer Risk. Because GTSS uses party shares as tracing inputs, a compromised tracer with access to all shares could recover the secret or fabricate evidence to frame honest parties. A robust system must ensure secrecy and non-imputability against the tracer. Since GTSS inherently requires share disclosure for tracing, the system naturally separates into *Standard Lifecycle* (including sharing and reconstruction phases), where parties retain their shares to preserve secrecy when no collusion occurs, and *Accountability Mechanism* (or tracing phase), where parties disclose their shares, trading secrecy for accountability. This separation introduces an additional challenge.

Challenge 3: The Tracing Trigger Problem. Because valid shares must be disclosed to the tracer upon a leakage claim, an adversary can submit fabricated evidence to trigger the tracing phase and collect parties' shares. A robust system must ensure that the tracing phase is initiated only when collusion has actually occurred. This requires a *testable trigger* predicate that initiates tracing only upon verified evidence of a functional leakage oracle.

Contributions. Our contributions are threefold:

- *Malicious Adversarial Model:* We propose a formal syntax for TSS-PV, defining public verifiability, traceability, and non-imputability against *verifiability-aware* perfect reconstruction boxes that can invoke all public verification predicates. We formally define a *testable trigger event* that initiates tracing only upon proof of a functional leakage oracle. Our work introduces verifiability-phase adversarial models that prior TSS schemes [10, 28] lack: we support a fully malicious dealer and malicious parties for all three verifiability properties, while traceability assumes an honest dealer, consistent with related work.
- *Specific Construction:* We propose an efficient (t, n) construction with linear public verification and quadratic dealer sharing. Adopting the GTSS mechanism, we sample $t-1$ dummy shares during the Sharing phase via an Anonymous Authenticated Channel (AAC) [4, 21], ensuring they are embedded into the public transcript and indistinguishable from real shares even to a box with a full network view. For efficient public verifiability, we propose a new Σ -protocol

for power-chain relations that enables share distribution without encryption or heavyweight zero-knowledge proofs.

- *Proof-of-Concept:* We implement a Curve25519 prototype (AVX2) and benchmark the end-to-end pipeline. Distribution checks scale linearly in n ; submission checks and reconstruction scale linearly in t ; tracing scales with the number of R queries, explicit in our analysis.

We clarify the adversarial scope and trust model as follows.

Standard Lifecycle: We model fully malicious dealers and parties in sharing and reconstruction phases; our verifiability definitions ensure detection of any malformed distribution.

Tracing Phase: Our analysis relies on the verifiability-aware perfect reconstruction box (formalized in Section 5): R is modeled as a deterministic oracle that behaves correctly on all valid inputs. Under this model, we achieve two guarantees. *Traceability* assumes honest share distribution, justified because a dealer intending to leak can do so directly, and our verifiability predicates provide a testable precondition before tracing proceeds. *Non-Imputability* holds against a malicious dealer: since the box's behavior is deterministic and defined solely by its embedded shares, a dealer cannot manufacture a box that frames an honest party.

Paper Structure. We show additional related work in Section 2. Section 3 introduces notation and complexity assumptions. Section 4 defines our TSS-PV syntax. Section 5 formalizes our security notions. Section 6 presents the proposed proof systems, system model, construction, and security analysis. Section 7 evaluates performance. Section 8 discusses limitations and future directions.

2 Additional Related Work

We survey additional related work to our research. Table 1 compares our TSS-PV with some closely related schemes.

Traceable Verifiable Secret Sharing. The recent TVSS [2] targets both verifiability and traceability but addresses only protocol-level misbehavior (compliant disclosures) rather than reconstruction-box leakage, leaving black-box threats undressed.

General Structure Traceable Secret Sharing. Farràs and Guiot [18] extended the black-box tracing framework to general access structures and proposed a concrete scheme supporting them; they also refined the definition of anonymous secret sharing and provided a construction under their strengthened notions. Recently, Goyal et al. [27] revisited TSS foundations and identified shortcomings in prior formulations regarding reconstruction box capabilities. While they achieve stronger traceability for general access structures, their model relies on dynamic share generation during tracing.

Anonymous Secret Sharing. Anonymous secret sharing [8, 30, 38] requires that shares reveal no information about the identity of the parties holding them. Bishop et al. [6] recently

Table 1: Comparison with related schemes. **PV**: public verifiability; **Box**: supports tracing under reconstruction-box leakage model; **Model**: assumption on box behavior (ϵ : ϵ -good, P: perfect); **NI**: non-imputability; **Assump.**: main cryptographic assumption;

Scheme	PV	Box	Model	NI	Assump.
GSS'21 [28]	$\times$	$\checkmark$	ϵ	$\checkmark$	OWF
BPR'24 [10]	$\times$	$\checkmark$	ϵ / P^a	$\checkmark$	OWF
GJP'25 [27]	$\times$	$\checkmark$	ϵ	$\checkmark$	iO/OWF ^b
BEMS'25 [2]	$\times^c$	$\times^d$	—	$\checkmark$	DDH/DLOG
GHL'22 [24]	$\checkmark$	$\times$	—	—	LWE
TSS-PV	$\checkmark$	$\checkmark$	P	$\checkmark$	DDH/ ℓ -DLOG

^aSSS/BSS-specific tracing achieves ϵ -good; GTSS requires perfect box and trusted tracer.

^bSub-exponential iO + OWF for $\emptyset$ -strong traceability; OWF-only construction achieves weaker guarantee.

^cParticipant-verifiable (VSS), not publicly verifiable.

^dTraces protocol-level misbehavior (compliant disclosures), not reconstruction-box leakage.

strengthened this notion by introducing Fully Anonymous Secret Sharing (FASS).

Post-Quantum PVSS. While early works established the foundations of PVSS [12, 19, 32, 37], recent research focuses on scalability and post-quantum security. Gentry et al. [24] and Minh et al. [31] propose constructions based on Learning With Errors (LWE) to achieve post-quantum security in the standard model.

Traitor Tracing. Traitor tracing [13, 15, 17, 23, 25, 26, 39] identifies parties whose secret decryption keys were combined into a pirate decoder. While structurally related, TSS-PV differs fundamentally: the secret is shared rather than individually encrypted, and tracing targets the sharing coalition rather than individual key holders.

3 Preliminary

This section introduces notation, complexity assumptions, and the standard NIZK proof system underlying our construction.

3.1 Notations and Paper Structure

We use the following notations throughout the paper. For sets: $[n]$ denotes $\{1, \dots, n\}$ for $n \in \mathbb{N}$; $\{x_i\}_{i \in [n]}$ abbreviates $\{x_1, \dots, x_n\}$, and $\{x_i\}_{i \in T}$ the subset indexed by T ; $\{a, b\}$ is an unordered set ($\{a, b\} = \{b, a\}$), while (a, b) is an ordered tuple ($(a, b) \neq (b, a)$). We write $\vec{x} = (x_1, \dots, x_n)$ for vectors and $\vec{x} \in S^n$ to indicate $\vec{x}$ has n components from S . For operations: $x \leftarrow F(y)$ denotes computing F on input y and assigning the result to x ; $r \leftarrow S$ means sampling r uniformly at random from S ; $A := B$ defines A to be B . For algorithms: A^O indicates A has oracle access to O ; $F(\mathcal{P}(x) \leftrightarrow \mathcal{D}(y)) \rightarrow o$ describes an interactive protocol F between party $\mathcal{P}$ (input x) and $\mathcal{D}$ (input

y), producing output o .

3.2 Complexity Assumptions

Our construction works under the ℓ -DLOG and DDH assumptions described below.

Definition 3.1 (ℓ -DLOG [22]). Given an integer ℓ , a group $\mathbb{G}$ of prime order q , a generator g , and a set $\{g^x, \dots, g^{x^\ell}\}$, we say that the ℓ -DLOG assumption holds in $\mathbb{G}$ if the probability that an adversary $\mathcal{A}$ outputs $x \in \mathbb{Z}_q$ is negligible. More formally:

$$\Pr[x' \leftarrow \mathcal{A}(g, g^x, \dots, g^{x^\ell}) : x' = x] = \text{negl}(\lambda),$$

where $\text{negl}(\cdot)$ is a negligible function and $\lambda \in \mathbb{N}$ is the security parameter.

Definition 3.2 (DDH [9]). Given a group $\mathbb{G}$ of prime order q , a generator g , and three elements $g^a, g^b, T \in \mathbb{G}$, we say that the DDH assumption holds in $\mathbb{G}$ if it is infeasible for an adversary $\mathcal{A}$ to distinguish between $T = g^{ab}$ and $T = g^z$, where $z \leftarrow \mathbb{Z}_q$. More formally:

$$\left| \Pr[\mathcal{A}(g, g^a, g^b, g^{ab}) = 1] - \Pr[\mathcal{A}(g, g^a, g^b, g^z) = 1] \right| = \text{negl}(\lambda),$$

where $\text{negl}(\cdot)$ is a negligible function, and $\lambda \in \mathbb{N}$ is the security parameter.

3.3 NIZK Proof

Our construction requires Schnorr-type Σ -protocols for proving linear relations among discrete logarithms DL-REP [11], rendered non-interactive via the Fiat–Shamir transform [20] in the random-oracle model (ROM).

Specifically, given a public statement $\vec{Y} = (Y_1, \dots, Y_u) \in \mathbb{G}^u$ and public bases $G = (\vec{g}_1, \dots, \vec{g}_u)$ where each $\vec{g}_i = (g_{i,1}, \dots, g_{i,l}) \in \mathbb{G}^l$, we prove that a witness $\vec{w} = (w_1, \dots, w_l) \in \mathbb{Z}_q^l$ is valid for the relation:

$$\mathcal{R}^{\text{LIN}} = \left\{ (\vec{Y}, G; \vec{w}) \mid \forall i \in [u] : Y_i = \prod_{j=1}^l g_{i,j}^{w_j} \right\}$$

Let $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ be a random oracle. The protocol $\Pi^{\text{NIZK}} = (\text{NIZK.Prf}, \text{NIZK.Vr})$ for $\mathcal{R}^{\text{LIN}}$ is defined as:

- **NIZK.Prf**($\vec{Y}, G; \vec{w}$) $\rightarrow \pi$: Sample $\vec{r} \leftarrow \mathbb{Z}_q^l$ and compute:

$$\forall i \in [u] : A_i \leftarrow \prod_{j=1}^l g_{i,j}^{r_j}, \quad c \leftarrow H(\vec{Y}, G, \vec{A}),$$

$$\forall j \in [l] : z_j \leftarrow r_j + c \cdot w_j,$$

where each A_i is considered a commitment in the Σ -protocol. Output $\pi := (c, \vec{z})$.

- $\text{NIZK.Vr}(\vec{Y}, G, \pi) \rightarrow \{0, 1\}$: Parse $\pi = (c, \vec{z})$, compute $A'_i \leftarrow Y_i^{-c} \cdot \prod_{j=1}^l g_{i,j}^{z_j}$ for all $i \in [u]$, and return 1 iff $c = H(\vec{Y}, G, \vec{A}')$.

Informally, a secure NIZK proof system satisfies *completeness* (any honest prover holding a valid witness can convince the verifier), *zero-knowledge* (there exists a simulator that, given only the statement, can generate proofs indistinguishable from real ones), and *soundness* (no PPT adversary can produce an accepting proof for a false statement except with negligible probability).

Theorem 3.1 (Security of NIZK). The DL-REP NIZK proof system is perfectly complete, zero-knowledge, and sound in the ROM [11, 33].

These NIZK proof systems can also be shown to be proofs of knowledge (PoK), i.e., they satisfy *knowledge-soundness* in the ROM: any PPT prover that produces an accepting proof admits an efficient extractor that outputs a corresponding witness, except with negligible probability.

4 Syntax

This section defines the syntax of TSS-PV and verifiability-aware reconstruction box R . The core syntax combines elements of TSS [10] and PVSS [24, 37].

TSS-PV Syntax. Given public parameters $\text{pp} := (1^\lambda, n, t, \rho)$, where $\rho \in \{0, 1\}^\lambda$ denotes a correlation string (e.g., ρ specifies the basis elements for our exponent-based commitments), a t -out-of- n Traceable Secret Sharing with Public Verifiability (TSS-PV) scheme consists of seven PPT algorithms:

$\text{TSS-PV} = (\text{Share}, \text{Rec}, \text{VerSD}, \text{VerSS}, \text{VerRS}, \text{Trace}^R, \text{TrVer}^R)$.

As in Boneh et al. [10], we adopt a symmetric formulation in which the dealer interacts independently with each index $i \in [n]$ by running Share , rather than sampling all n shares in a single vector. All algorithms are executed relative to a fixed public environment determined by $(1^\lambda, n, t, \rho)$; construction-specific public elements (e.g., an algebraic domain and correlation material derived from ρ) are fixed once and omitted from interfaces for readability. We define the algorithms below and omit pp from inputs when clear.

- $\text{Share}(\mathcal{P}_i(\text{ps}_i) \leftrightarrow \mathcal{D}(\text{ds})) \rightarrow (\text{op}_i, \text{od}_i, T_i)$: an interactive protocol between the dealer $\mathcal{D}$, holding dealer state ds (which includes the secret S and dealer randomness ω), and party $\mathcal{P}_i$, holding party state ps_i . It produces:
 - Party output $\text{op}_i = (\text{sh}_i, \text{ptk}_i, \text{pvk}_i)$, where sh_i is the share, ptk_i and pvk_i are the party tracing and verification keys.
 - Dealer output $\text{od}_i = (\text{dtk}_i, \text{dvk}_i)$, where dtk_i and dvk_i are the dealer tracing and verification keys.
 - T_i is the public transcript generated during sharing.

We denote the combined tracing and verification keys by $\text{tk}_i := (\text{ptk}_i, \text{dtk}_i)$ and $\text{vk}_i := (\text{pvk}_i, \text{dvk}_i)$, and write $\text{TK} := \{\text{tk}_i\}_{i \in [n]}$ and $\text{VK} := \{\text{vk}_i\}_{i \in [n]}$ for the global key sets.

- $\text{Rec}(\text{SH}) \rightarrow S' / \perp$: On input any size- t set of valid shares $\text{SH} \subseteq \{\text{sh}_i\}_{i \in [n]}$, outputs a candidate secret S' . If $|\text{SH}| < t$, it returns $\perp$. Note, validity is defined by VerSS below.
- $\text{VerSD}(\text{T}) \rightarrow \{0, 1\}$: On input $\text{T} = \{T_i\}_{i \in [n]}$, this function checks if the dealer correctly distributed shares.
- $\text{VerSS}(\text{sh}_i, \text{T}) \rightarrow \{0, 1\}$: On input sh_i and $\text{T} = \{T_i\}_{i \in [n]}$, this function verifies if $\mathcal{P}_i$ submitted its share sh_i .
- $\text{VerRS}(S', \text{T}) \rightarrow \{0, 1\}$: This function takes as input S' and $\text{T} = \{T_i\}_{i \in [n]}$, then it verifies if S' is the shared secret.
- $\text{Trace}^R(\text{TK}, \text{T}) \rightarrow (C', \pi_{\text{trace}})$: The tracing function with oracle access to a reconstruction box R takes as input $\text{TK} = \{\text{tk}_i\}_{i \in [n]}$, and $\text{T} = \{T_i\}_{i \in [n]}$, outputting an accused index set $C' \subseteq [n]$ and a tracing proof π_{trace} .
- $\text{TrVer}^R(\text{VK}, \text{T}, C', \pi_{\text{trace}}) \rightarrow \{0, 1\}$: A public judging function with black-box access to the same reconstruction box R used by Trace^R . On input $\text{VK} = \{\text{vk}_i\}_{i \in [n]}$, $\text{T} = \{T_i\}_{i \in [n]}$, C' , and π_{trace} (both output by Trace^R), it checks if the accusation produced by Trace^R is correct.

Reconstruction Box Syntax. A reconstruction box R is generated by a coalition of parties (indexed by a subset $C \subset [n]$) of size f , where $0 < f < t$. The box embeds the coalition's shares internally and may invoke Rec and the public predicates, but its external interface is restricted to the following black-box query:

- $R(\text{SH}) \rightarrow S' / \perp$: on input a set of candidate shares SH , the box outputs either a secret S' or $\perp$. Note, we treat any output outside the secret space as $\perp$.

5 Security Notion

All security notions are defined for a fixed scheme $\text{TSS-PV} = (\text{Share}, \text{Rec}, \text{VerSD}, \text{VerSS}, \text{VerRS}, \text{Trace}^R, \text{TrVer}^R)$ over party secret space $\mathcal{PS} = \{\mathcal{PS}_\lambda\}_{\lambda \in \mathbb{N}}$, dealer secret space $\mathcal{S} = \{\mathcal{S}_\lambda\}_{\lambda \in \mathbb{N}}$, dealer randomness space $\mathcal{O} = \{\mathcal{O}_\lambda\}_{\lambda \in \mathbb{N}}$, and share space $\mathcal{SH} = \{\mathcal{SH}_\lambda\}_{\lambda \in \mathbb{N}}$.

We adopt the static corruption model: $\mathcal{A}$ declares all corrupted parties at the onset of each experiment and may deviate arbitrarily from the protocol. TSS-PV is a symmetric secret sharing scheme in which all shares are identically distributed, so observing protocol executions adaptively before selecting corrupted parties confers no additional advantage [10].

Our correctness, dealer secrecy, and the verifiability notions adapt the standard VSS/PVSS framework [10, 19, 37]. Party secrecy (Def. 5.6), verifiability-aware reconstruction box (Def. 5.8), the tracing trigger event $\mathcal{T}$ (Def. 5.9), and the non-imputability definition under a verifiability-aware adversary

(Def. 5.11) are introduced in this work. The perfect reconstruction box (Def. 5.8) extends Boneh et al. [10] by explicitly granting R access to all public predicates.

5.1 Correctness

Informally, correctness requires that if all parties are honest then all proofs and predicates pass and any subset of t shares recovers the secret.

Definition 5.1 (Correctness). Let $(pp, \Sigma := \{sh_i\}_{i \in [n]}, T := \{T_i\}_{i \in [n]})$ be the output of experiment $\mathbf{ExpCor}_{TSS-PV}^{n,t}(\lambda)$ as per Fig. 1. Let $\epsilon = \epsilon(\lambda) \in [0, 1]$ be a function of the security parameter λ . We say the scheme is ϵ -correct if for every $n \in \mathbb{N}$, every $0 < t \leq n$, the following probability is at least $1 - \epsilon$, where the probability is taken over the coins of Share and $\mathbf{ExpCor}_{TSS-PV}^{n,t}(\lambda)$ internal randomness.

$$\Pr \left[\begin{array}{l} (\text{VerSD}(T) = 1) \\ \wedge (\forall sh \in \Sigma : \text{VerSS}(sh, T) = 1) \\ \wedge \exists S' \in \mathcal{S}_\lambda \text{ s.t. } \forall SH \subseteq \Sigma, |SH| = t : \\ \quad \text{Rec}(SH) = S' \wedge \text{VerRS}(S', T) = 1 \\ \wedge (\forall SH' \subseteq \Sigma, |SH'| < t : \text{Rec}(SH') = \perp) \end{array} \right]$$

Experiment $\mathbf{ExpCor}_{TSS-PV}^{n,t}(\lambda)$

1. $\rho \leftarrow \{0, 1\}^\lambda, pp := (1^\lambda, n, t, \rho);$
2. $\{ps_i\}_{i \in [n]} \leftarrow \mathcal{PS}_\lambda, S \leftarrow \mathcal{S}_\lambda, \omega \leftarrow \Omega_\lambda, ds := (S, \omega);$
3. $\forall i \in [n] : (op_i, od_i, T_i) \leftarrow \text{Share}(\mathcal{P}_i(ps_i) \leftrightarrow \mathcal{D}(ds)),$
where $op_i = (sh_i, ptk_i, pvk_i), od_i = (dtk_i, dvk_i);$
4. **return** $(pp, \Sigma := \{sh_i\}_{i \in [n]}, T := \{T_i\}_{i \in [n]}).$

Figure 1: The correctness experiment for TSS-PV. Dealer and all parties are honest.

5.2 Verifiability

We define three verification properties.

Distribution verifiability (VerSD). We say TSS-PV has *verifiable distribution* if, whenever $\text{VerSD}(T) = 1$, any two size- t subsets of shares that individually pass VerSS reconstruct the same secret. This property captures security against a *malicious dealer*: a cheating dealer cannot cause different coalitions to recover different secrets while the transcript appears valid.

Definition 5.2 (VerSD). Let $\mathbf{ExpVSD}_{\mathcal{A}, TSS-PV}(\lambda)$ be the experiment in Fig. 2. We say TSS-PV has *verifiable distribution* if for every PPT adversary $\mathcal{A}$, the following advantage is negligible for any security parameter $\lambda \in \mathbb{N}$.

$$\text{Adv}_{\mathcal{A}, TSS-PV}^{\text{vsd}}(\lambda) := \Pr [\mathbf{ExpVSD}_{\mathcal{A}, TSS-PV}(\lambda) = 1],$$

where the probability is taken over the random coins of $\mathcal{A}$ and $\mathbf{ExpVSD}_{\mathcal{A}, TSS-PV}(\lambda)$ internal randomness.

Experiment $\mathbf{ExpVSD}_{\mathcal{A}, TSS-PV}(\lambda)$

1. $(n, t, \text{state}) \leftarrow \mathcal{A}(1^\lambda)$, where $0 < t \leq n;$
2. $\rho \leftarrow \{0, 1\}^\lambda, pp := (1^\lambda, n, t, \rho);$
3. $(T := \{T_i\}_{i \in [n]}, \Sigma := \{sh_i\}_{i \in [n]}, I_0, I_1) \leftarrow \mathcal{A}(\text{state}, pp);$
4. $\mathcal{V}(T, \Sigma) := \{sh_i \in \Sigma : \text{VerSS}(sh_i, T) = 1\};$
5. **if** $(|\mathcal{V}(T, \Sigma)| < t) \vee (|I_0| \neq t) \vee (|I_1| \neq t) \vee (I_0 = I_1) \vee (I_0 \not\subseteq \{i \in [n] : sh_i \in \mathcal{V}(T, \Sigma)\}) \vee (I_1 \not\subseteq \{i \in [n] : sh_i \in \mathcal{V}(T, \Sigma)\})$
then return 0;
6. $S_b \leftarrow \text{Rec}(SH_b)$, where $SH_b := \{sh_i : i \in I_b\}$ for $b \in \{0, 1\};$
7. **if** $(\text{VerSD}(T) = 1) \wedge (S_0 \neq S_1)$ **then return** 1;
8. **return** 0.

Figure 2: Distribution verifiability experiment. The adversary simulates the dealer and all parties.

Share Submission Verifiability (VerSS). This property captures security against *malicious parties* when the dealer is honest. VerSS ensures two guarantees: (i) *Binding*: no adversary can produce two distinct valid submissions for the same party index; and (ii) *Consistency*: any t valid submissions reconstruct to the dealer's original secret S .

Definition 5.3 (VerSS). Let $\mathbf{ExpVSS}_{\mathcal{A}, TSS-PV}(\lambda)$ be the experiment in Fig. 3. We say TSS-PV has *verifiable submission* if for every PPT adversary $\mathcal{A}$, the following advantage is negligible for any security parameter $\lambda \in \mathbb{N}$.

$$\text{Adv}_{\mathcal{A}, TSS-PV}^{\text{vss}}(\lambda) := \Pr [\mathbf{ExpVSS}_{\mathcal{A}, TSS-PV}(\lambda) = 1],$$

where the probability is taken over the random coins of $\mathcal{A}$, Share, and $\mathbf{ExpVSS}_{\mathcal{A}, TSS-PV}(\lambda)$ internal randomness.

Reconstructed Secret Verifiability (VerRS). The predicate VerRS provides *public binding* for the reconstructed secret: for any transcript T with $\text{VerSD}(T) = 1$, there is at most one $S' \in \mathcal{S}_\lambda$ satisfying $\text{VerRS}(S', T) = 1$. Combined with correctness, this implies that in honest executions S' equals the dealer's secret. A malicious dealer may publish malformed commitments so that $\text{VerRS}(S', T) = 0$ for every S' ; we treat this as denial-of-service rather than a security violation, analogous to a dealer who refuses to share a well-formed secret.

Definition 5.4 (VerRS). Let $\mathbf{ExpVRS}_{\mathcal{A}, TSS-PV}(\lambda)$ be the experiment in Fig. 4. We say TSS-PV has *verifiable secret reconstruction* if for every PPT adversary $\mathcal{A}$, the following advantage is negligible for any security parameter $\lambda \in \mathbb{N}$.

$$\text{Adv}_{\mathcal{A}, TSS-PV}^{\text{vrec}}(\lambda) := \Pr [\mathbf{ExpVRS}_{\mathcal{A}, TSS-PV}(\lambda) = 1],$$

where the probability is taken over random coins of $\mathcal{A}$ and internal randomness of $\mathbf{ExpVRS}_{\mathcal{A}, TSS-PV}(\lambda)$.

Experiment **ExpVSS** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$}

1. $(n, t, \text{state}_1) \leftarrow \mathcal{A}(1^\lambda)$, where $0 < t \leq n$;
 2. $\rho \leftarrow \$ \{0, 1\}^\lambda$, $\text{pp} := (1^\lambda, n, t, \rho)$;
 3. $\text{state}_2 \leftarrow \mathcal{A}(\text{state}_1, \text{pp})$;
 4. $S \leftarrow \$ \mathcal{S}_\lambda$, $\omega \leftarrow \$ \Omega_\lambda$, $\text{ds} := (S, \omega)$;
 5. $\forall i \in [n] : ((\cdot), (\text{dtk}_i, \text{dvk}_i), T_i) \leftarrow \text{Share}(\mathcal{A} \leftrightarrow \mathcal{D}(\text{ds}))$;
 6. $T := \{T_i\}_{i \in [n]}$;
 7. $(\text{SH}, i^*, \text{sh}_{i^*}, \text{sh}'_{i^*}) \leftarrow \mathcal{A}(\text{state}_2, T)$, where $|\text{SH}| = t, \text{SH} \subseteq \mathcal{SH}_\lambda$;
 8. $S' \leftarrow \text{Rec}(\text{SH})$;
 9. $b_0 \leftarrow (\text{VerSS}(\text{sh}_{i^*}, T) = \text{VerSS}(\text{sh}'_{i^*}, T) = 1) \wedge (\text{sh}_{i^*} \neq \text{sh}'_{i^*})$;
 10. $b_1 \leftarrow (\forall \text{sh} \in \text{SH} : \text{VerSS}(\text{sh}, T) = 1) \wedge (S' \neq S)$;
 11. **if** $b_0 \vee b_1$ **then return 1**;
 12. **return 0**;
-

Figure 3: The experiment of share-submission verifiability. The adversary simulates malicious parties.

Experiment **ExpVRS** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$}

1. $(n, t, \text{state}) \leftarrow \mathcal{A}(1^\lambda)$, where $0 < t \leq n$;
 2. $\rho \leftarrow \$ \{0, 1\}^\lambda$, $\text{pp} := (1^\lambda, n, t, \rho)$;
 3. $(T := \{T_i\}_{i \in [n]}, S_0, S_1) \leftarrow \mathcal{A}(\text{state}, \text{pp})$, where $S_0, S_1 \in \mathcal{S}_\lambda$ and $S_0 \neq S_1$;
 4. **if** $(\text{VerRS}(S_0, T) = 1) \wedge (\text{VerRS}(S_1, T) = 1)$ **then return 1**;
 5. **return 0**;
-

Figure 4: VerRS experiments. The adversary plays the role of the dealer and all parties.

All three predicates are standalone: each re-checks global consistency internally, so no predicate assumes prior execution of another. The algorithmic realization of this design appears in Remark 6.2.

5.3 Secrecy

Informally, secrecy consists of dealer secrecy and party secrecy. Dealer secrecy requires that an adversary controlling at most $t - 1$ parties cannot recover the shared secret beyond random guessing. Party secrecy requires that each party's share remains private to its owner, so that even the dealer cannot recover it.

Definition 5.5 (Dealer Secrecy). Let $\text{ExpDSec}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$ be the experiment defined in Fig. 5, we say TSS-PV satisfies *dealer secrecy* if for every PPT adversary $\mathcal{A}$ and every security parameter $\lambda \in \mathbb{N}$, it holds that the following advantage is negligible in λ .

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{dsec}}(\lambda) := \left| \Pr[\text{ExpDSec}_{\mathcal{A}, \text{TSS-PV}}(\lambda) = 1] - \frac{1}{|\mathcal{S}_\lambda|} \right|,$$

Experiment **ExpDSec** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$}

1. $(n, t, \text{state}_1) \leftarrow \mathcal{A}(1^\lambda)$, where $0 < t \leq n$;
 2. $\{\text{ps}_i\}_{i=t}^n \leftarrow \$ \mathcal{PS}_\lambda$;
 3. $\rho \leftarrow \$ \{0, 1\}^\lambda$, $\text{pp} := (1^\lambda, n, t, \rho)$;
 4. $\text{state}_2 \leftarrow \mathcal{A}(\text{state}_1, \text{pp})$;
 5. $S \leftarrow \$ \mathcal{S}_\lambda$, $\omega \leftarrow \$ \Omega_\lambda$, $\text{ds} := (S, \omega)$;
 6. **for** $i=1, \dots, t-1$: $((\cdot), (\text{dtk}_i, \text{dvk}_i), T_i) \leftarrow \text{Share}(\mathcal{A} \leftrightarrow \mathcal{D}(\text{ds}))$;
 7. **for** $i=t, \dots, n$:
 $((\text{sh}_i, \text{ptk}_i, \text{pvk}_i), (\text{dtk}_i, \text{dvk}_i), T_i) \leftarrow \text{Share}(\mathcal{P}_i(\text{ps}_i) \leftrightarrow \mathcal{D}(\text{ds}))$;
 8. $S^* \leftarrow \mathcal{A}(\text{state}_2, \{T_i\}_{i \in [n]})$;
 9. **if** $S = S^*$ **then return 1**;
 10. **return 0**.
-

Figure 5: The dealer secrecy experiment. The adversary $\mathcal{A}$ controls $t - 1$ parties during the sharing phase.

where the probability is taken over random coins of $\mathcal{A}$ and internal randomness of $\text{ExpDSec}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$.

Definition 5.6 (Party secrecy). Let $\text{ExpPSec}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$ be the experiment defined in Fig. 6, we say TSS-PV satisfies *party secrecy* if for every PPT adversary $\mathcal{A}$ and every security parameter $\lambda \in \mathbb{N}$, the following advantage is negligible in λ .

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{psec}}(\lambda) := \left| \Pr[\text{ExpPSec}_{\mathcal{A}, \text{TSS-PV}}(\lambda) = 1] - \frac{1}{|\mathcal{SH}_\lambda|} \right|,$$

where the probability is taken over random coins of $\mathcal{A}$ and internal randomness of $\text{ExpPSec}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$.

Experiment **ExpPSec** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$}

1. $(n, t, i^*, \{\text{ps}_j\}_{j \in [n] \setminus \{i^*\}}, \text{state}_1) \leftarrow \mathcal{A}(1^\lambda)$, where $0 < t \leq n$ and $i^* \in [n]$;
 2. $\rho \leftarrow \$ \{0, 1\}^\lambda$, $\text{pp} := (1^\lambda, n, t, \rho)$, $\text{ps}_{i^*} \leftarrow \$ \mathcal{PS}_\lambda$;
 3. $\text{state}_2 \leftarrow \mathcal{A}(\text{state}_1, \text{pp})$;
 4. $((\text{sh}_{i^*}, \text{ptk}_{i^*}, \text{pvk}_{i^*}), (\cdot), T_{i^*}) \leftarrow \text{Share}(\mathcal{P}_{i^*}(\text{ps}_{i^*}) \leftrightarrow \mathcal{A})$;
 5. $\hat{\text{sh}}_{i^*} \leftarrow \mathcal{A}(\text{state}_2, T_{i^*})$;
 6. **if** $\hat{\text{sh}}_{i^*} = \text{sh}_{i^*}$ **then return 1 else return 0**;
-

Figure 6: The party secrecy experiment for TSS-PV. The adversary $\mathcal{A}$ simulates the dealer and $n - 1$ parties.

Remark 5.1 (Remarks on Secrecy). We make the following remarks on the secrecy experiments.

- **The Advantage Term.** We subtract $1/|\mathcal{S}_\lambda|$ and $1/|\mathcal{SH}_\lambda|$ from the adversary advantages $\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{dsec}}(\lambda)$ and $\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{psec}}(\lambda)$, respectively, to discount the trivial attack where $\mathcal{A}$ guesses the secret $S \in \mathcal{S}_\lambda$ and $\text{sh} \in \mathcal{SH}_\lambda$ (which

naturally succeeds with probability $1/|\mathcal{S}_\lambda|$ and $1/|\mathcal{SH}_\lambda|$, respectively).

- **Computational Secrecy.** Our scheme achieves computational dealer secrecy rather than perfect (information-theoretic) dealer secrecy. This is an inherent consequence of public reconstruction verifiability: the predicate VerRS binds the secret S to the public transcript T via a commitment that is computationally hiding under DDH but not information-theoretically hiding. An unbounded adversary could enumerate all candidates $S' \in \mathcal{S}_\lambda$ and check $\text{VerRS}(S', T)$ to recover S .

5.4 Traceability

Traceability requires that if a coalition of $f < t$ parties creates a reconstruction box that reconstructs the secret upon receiving additional $t - f$ shares, then the tracing algorithm can identify all of the leaking parties from reconstruction-box access. Because traceability only makes sense if the dealer distributed shares correctly so that any subset of t valid shares reconstruct the secret, we first define a distribution predicate as follows.

Definition 5.7 (Distribution Predicate). Let $(\Sigma := \{\text{sh}_i\}_{i \in [n]}, T := \{T_i\}_{i \in [n]})$ be the output of running n instances of Share . Let $\mathcal{V}(T, \Sigma) := \{\text{sh} \in \Sigma : \text{VerSS}(\text{sh}, T) = 1\}$ be the set of valid shares. We define a correct distribution $\Phi_{\text{dist}}(T, \Sigma)$ as

$$\Phi_{\text{dist}}(T, \Sigma) := \left[\begin{array}{l} |\mathcal{V}(T, \Sigma)| \geq t \\ \wedge \exists S' \in \mathcal{S}_\lambda \text{ s.t. } \forall \text{SH} \subseteq \mathcal{V}(T, \Sigma), |\text{SH}| = t : \\ \quad \text{Rec}(\text{SH}) = S' \wedge \text{VerRS}(S', T) = 1 \end{array} \right].$$

Remark 5.2 (Remark on Def. 5.7.). By the binding property of VerRS (Def. 5.4), the secret S' in Φ_{dist} is unique whenever $\Phi_{\text{dist}}(T, \Sigma) = 1$. Conversely, $\Phi_{\text{dist}}(T, \Sigma) = 0$ indicates at least one of the three conjuncts in the definition failed.

Then, under the correct distribution ($\Phi_{\text{dist}}(T, \Sigma) = 1$), we define a perfect reconstruction box that is verifiability-aware.

Definition 5.8 (Verifiability-aware Perfect Reconstruction Box). Fix (T, Σ) with $\Phi_{\text{dist}}(T, \Sigma) = 1$ and unique S' . An oracle R is a verifiability-aware perfect reconstruction box for a coalition $C \subseteq [n]$ with leakage $f = |C|$ if its internal state contains exactly the shares $E = \{\text{sh}_i : i \in C\} \subseteq \mathcal{V}(T, \Sigma)$ and the following holds for all $\text{SH} \subseteq \mathcal{V}(T, \Sigma)$:

- **(Valid inputs)** If $|\text{SH}| = t - f$ and $\text{SH} \cap E = \emptyset$, then

$$\Pr[\text{VerRS}(R(\text{SH}), T) = 1] = 1$$

and moreover $R(\text{SH}) = S'$ with probability 1.

- **(Invalid inputs)** Otherwise,

$$\Pr[R(\text{SH}) = \perp \vee \text{VerRS}(R(\text{SH}), T) = 0] = 1.$$

The probability is over the internal randomness of R .

Remark 5.3 (Remark on Def. 5.8). In this model, the oracle R has access to the public transcript T and all verification predicates of TSS-PV (VerSD , VerSS , and VerRS); it may invoke these predicates internally to validate input shares before attempting reconstruction.

Given a verifiability-aware perfect reconstruction box as per Def. 5.8, we define the tracing trigger event $\mathcal{T}$ as follows.

Definition 5.9 (Tracing Trigger Event). Let $\Phi_{\text{dist}}(T, \Sigma) = 1$ as per Def. 5.7 and let R be a verifiability-aware perfect reconstruction box as in Def. 5.8. For a coalition $C \subseteq [n]$ of size f , we say the tracing trigger event $\mathcal{T}(T, \Sigma, R; C)$ occurs if and only if there exists $\text{SH} \subseteq \mathcal{V}(T, \Sigma)$ with $|\text{SH}| = t - f$ and $\text{SH} \cap \{\text{sh}_i : i \in C\} = \emptyset$ such that $\text{VerRS}(R(\text{SH}), T) = 1$.

Remark 5.4 (Leakage Evidence). Def. 5.9 defines the trigger event $\mathcal{T}$ based on the existence of a verifiability-aware perfect reconstruction box. In the concrete protocol, the dealer verifies $\mathcal{T}$ by probing the box with dummy shares. The probe demonstrates functionality on the test set; our formal model (which assumes honest sharing) treats this as sufficient evidence of leakage.

We now define traceability as follows.

Definition 5.10 (Traceability). Let $\text{ExpTrace}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$ be the experiment of Fig. 7. Let Φ_{dist} be the distribution predicate as per Def. 5.7, R be the verifiability-aware perfect reconstruction box as per Def. 5.8, and $\mathcal{T}$ be the tracing trigger event as per Def. 5.9. We say that TSS-PV satisfies *traceability* if the following advantage is negligible in λ for every PPT adversary $\mathcal{A}$.

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{trace}}(\lambda) := \left| \Pr[\text{ExpTrace}_{\mathcal{A}, \text{TSS-PV}}(\lambda) = 1] - \frac{1}{|\mathcal{S}_\lambda|} \right|,$$

where the probability is over $\text{ExpTrace}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$ internal randomness, Share , R , and $\mathcal{A}$.

Remark 5.5 (Remark on Def. 5.10). We clarify our definition of Traceability as follows.

- **Adversarial Model.** The traceability experiment (Def. 5.10) assumes honest share generation (thus, TK and VK), and honest trigger activation by the dealer. This scopes the definition to capturing *correctness* of tracing (leakers are identified) rather than *liveness* (tracing will occur). Our construction achieves additional verifiability properties beyond what the definition requires.
- **Adversarial Advantage.** As in [10], we discount the baseline $1/|\mathcal{S}_\lambda|$ to remove the trivial strategy where a box R ignores its inputs and returns a uniformly random secret; such a box passes VerRS with probability exactly $1/|\mathcal{S}_\lambda|$, independent of the shares.

Experiment **ExpTrace** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$}

1. $(n, t, f, \text{state}) \leftarrow \mathcal{A}(1^\lambda)$, where $0 < f < t \leq n$;
 2. $\rho \leftarrow \$ \{0, 1\}^\lambda$, $\text{pp} := (1^\lambda, n, t, \rho)$, $S \leftarrow \$ \mathcal{S}_\lambda$, $\omega \leftarrow \$ \Omega_\lambda$,
 $\text{ds} := (S, \omega)$, $\{\text{ps}_i\}_{i \in [n]} \leftarrow \$ \mathcal{PS}_\lambda$;
 3. $\forall i \in [n] : (\text{op}_i, \text{od}_i, T_i) \leftarrow \text{Share}(\mathcal{P}_i(\text{ps}_i) \leftrightarrow \mathcal{D}(\text{ds}))$,
where $\text{op}_i = (\text{sh}_i, \text{ptk}_i, \text{pvk}_i)$, $\text{od}_i = (\text{dtk}_i, \text{dvk}_i)$;
 4. set $\text{TK} := \{\text{ptk}_i, \text{dtk}_i\}_{i \in [n]}$, $\text{VK} := \{(\text{pvk}_i, \text{dvk}_i)\}_{i \in [n]}$,
 $\Sigma := \{\text{sh}_i\}_{i \in [n]}$, and $\text{T} := \{T_i\}_{i \in [n]}$;
 5. $(C, R) \leftarrow \mathcal{A}(\text{state}, \text{pp}, \{\text{op}_i\}_{i \in [n]}, \text{T})$, where $|C| = f$ and $C \subset [n]$;
 6. **if** $\neg \mathcal{T}(\text{T}, \Sigma, R; C)$ **then return 0**;
 7. $(C', \pi_{\text{trace}}) \leftarrow \text{Trace}^R(\text{TK}, \text{T})$;
 8. **if** $(C' = C) \wedge (\text{TrVer}^R(\text{VK}, \text{T}, C', \pi_{\text{trace}}) = 1)$ **then return 0**;
 9. **return 1**;
-

Figure 7: Traceability experiment. $\mathcal{A}$ simulates f leakers.

5.5 Non-imputability

Informally, non-imputability indicates that an honest party cannot be wrongly accused of leaking its share.

Definition 5.11 (Non-imputability). Let **ExpNI** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$} be the experiment in Fig. 8. Let R be the verifiability-aware perfect reconstruction box as per Def. 5.8, and $\mathcal{T}$ be the tracing trigger event as per Def. 5.9. We say TSS-PV satisfies *non-imputability* if for every PPT adversary $\mathcal{A}$, and every $\lambda \in \mathbb{N}$, the following advantage is negligible.

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}(\lambda)}^{\text{ni}} := \Pr [\mathbf{ExpNI}_{\mathcal{A}, \text{TSS-PV}(\lambda)} = 1],$$

where the probability is over **ExpNI** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$} internal randomness, Share , R , and $\mathcal{A}$.

6 Construction

In this section, we propose our proof systems, the TSS-PV construction, and their security analysis.

6.1 Proof Systems

We propose NIZK instances and the PC-PoK as the fundamental building blocks for our TSS-PV construction.

6.1.1 NIZK Instances

We instantiate the NIZK from Section 3 for the following relations:

- **Per-party share proof π_i** : Given $\vec{Y} = (p_i, \text{cr}, \text{cs})$, bases $G = ((f_i, g), (h_1, 1_G), (1_G, h_2))$, witness $\vec{w} = (r, s)$, the proving relation is $p_i = f_i^r g^s \wedge \text{cr} = h_1^r \wedge \text{cs} = h_2^s$.

Experiment **ExpNI** _{$\mathcal{A}, \text{TSS-PV}(\lambda)$}

1. $(n, t, \text{state}_1) \leftarrow \mathcal{A}(1^\lambda)$, where $0 < t \leq n$;
 2. $\rho \leftarrow \$ \{0, 1\}^\lambda$, $\text{pp} := (1^\lambda, n, t, \rho)$;
 3. $(i^*, \text{state}_2) \leftarrow \mathcal{A}(\text{state}_1, \text{pp})$, where $i^* \in [n]$;
 4. $\text{ps}_{i^*} \leftarrow \$ \mathcal{PS}_\lambda$, $L := [n] \setminus \{i^*\}$;
 5. $((\text{sh}_{i^*}, \text{ptk}_{i^*}, \text{pvk}_{i^*}), (\cdot, T_{i^*}) \leftarrow \text{Share}(\mathcal{P}_{i^*}(\text{ps}_{i^*}) \leftrightarrow \mathcal{A})$;
 6. $(C, \pi_{\text{trace}}, \{(\text{sh}_\ell, \text{pvk}_\ell, T_\ell)\}_{\ell \in L}, \{\text{dvk}_\ell\}_{\ell \in [n]}, R) \leftarrow \mathcal{A}(\text{state}_2, T_{i^*})$;
 7. set $\text{T} := \{T_\ell\}_{\ell \in [n]}$, $\text{VK} := \{(\text{pvk}_\ell, \text{dvk}_\ell)\}_{\ell \in [n]}$, and $\Sigma := \{\text{sh}_\ell\}_{\ell \in [n]}$;
 8. **if** $\neg \mathcal{T}(\text{T}, \Sigma, R; C)$ **then return 0**;
 9. **if** $(i^* \in C) \wedge (\text{TrVer}^R(\text{VK}, \text{T}, C, \pi_{\text{trace}}) = 1)$ **then return 1**;
 10. **return 0**;
-

Figure 8: The Non-imputability experiment. The adversary $\mathcal{A}$ simulates the dealer $\mathcal{D}$ and $n - 1$ parties.

- **Reconstruction consistency proof ψ** : Given $\vec{Y} = (S', \text{cs})$, bases $G = ((g), (h_2))$, witness $\vec{w} = (s)$, the proving relation is $S' = g^s \wedge \text{cs} = h_2^s$. Note, the proof ψ is generated during the sharing phase (when $S' = S$ is known to the dealer) but becomes publicly verifiable only after S' is revealed (e.g., during reconstruction or tracing).

We denote by NIZK_π and NIZK_ψ the instantiations of Π^{NIZK} for the per-party proof π_i and the reconstruction proof ψ , respectively.

6.1.2 PC-PoK

In our construction, each party must prove knowledge of their evaluation point x_i such that $f_i = \prod_{j=1}^{t-1} \alpha_j^{x_i}$ for a public vector $\vec{\alpha} \in \mathbb{G}^{t-1}$. Standard DL-REP proofs [11] do not directly apply since the exponents $(x_i, x_i^2, \dots, x_i^{t-1})$ are powers of a single witness rather than independent values. We construct a specialized Σ -protocol (so-called PC-PoK) for this power-chain relation.

Formally, given the public parameters $\text{pp}^{\text{PC}} = (g, \vec{\alpha} \in \mathbb{G}^{t-1})$, we prove knowledge of a witness $x \in \mathbb{Z}_q$ for the power-chain relation:

$$\mathcal{R}^{\text{PC}} = \left\{ ((f, \vec{V}); x) \mid \begin{array}{l} V_0 = g \wedge \forall k \in [t-1] : V_k = (V_{k-1})^x \\ \wedge f = \prod_{k=1}^{t-1} \alpha_k^{x^k} \end{array} \right\}$$

We employ a strategy decomposing $\mathcal{R}^{\text{PC}}$ into two parallel Σ -protocols: *Track A* (Structure) uses chained DLEQ proofs [14] to establish $V_{j+1} = V_j^x$, and *Track B* (Consistency) uses a Schnorr-type multi-exponentiation proof [33] to link f with $\vec{V}$.

Let $H : \{0, 1\}^* \rightarrow \mathbb{Z}_q$ be a random oracle. The protocol $\Pi^{\text{PC}} = (\text{PC.Prf}, \text{PC.Vr})$ is defined as:

- $\text{PC.Prf}(\text{pp}^{\text{PC}}, x) \rightarrow (f, \vec{V}, \pi^{\text{PC}})$: The prover computes

$$\begin{aligned} V_0 &= g, \quad \forall k \in [t-1] : V_k = (V_{k-1})^x, \\ \vec{V} &:= (V_1, \dots, V_{t-1}), \quad f \leftarrow \prod_{k=1}^{t-1} \alpha_k^x, \\ \vec{r} &= (r_1, \dots, r_{t-1}) \leftarrow \mathbb{Z}_q^{t-1}, \quad \forall k \in [t-2] : A_k \leftarrow V_k^{r_1}, \\ \forall k \in [t-1] : B_k &\leftarrow g^{r_k}, \quad \hat{B} \leftarrow \prod_{k=1}^{t-1} \alpha_k^{r_k}, \\ c &\leftarrow H(\text{pp}^{\text{PC}}, f, \vec{V}, \{A_k\}_{k \in [t-2]}, \{B_k\}_{k \in [t-1]}, \hat{B}), \\ \forall k \in [t-1] : z_k &\leftarrow r_k + c \cdot x^k, \quad \vec{z} := (z_1, \dots, z_{t-1}), \end{aligned}$$

where $\{A_k\}_{k \in [t-2]}$, $\{B_k\}_{k \in [t-1]}$, and $\hat{B}$ are considered commitments. Output $\pi^{\text{PC}} := (c, \vec{z})$.

- $\text{PC.Vr}(\text{pp}^{\text{PC}}, f, \vec{V}, \pi^{\text{PC}}) \rightarrow \{0, 1\}$: The verifier parses $\pi^{\text{PC}} = (c, \vec{z})$ and computes

$$\begin{aligned} \forall k \in [t-2] : A'_k &\leftarrow V_k^{z_1} V_{k+1}^{-c}, \\ \forall k \in [t-1] : B'_k &\leftarrow g^{z_k} V_k^{-c}, \quad \hat{B}' \leftarrow \left(\prod_{k=1}^{t-1} \alpha_k^{z_k} \right) \cdot f^{-c}. \end{aligned}$$

The verifier accepts if $c = H(\text{pp}^{\text{PC}}, f, \vec{V}, \{A'_k\}_{k \in [t-2]}, \{B'_k\}_{k \in [t-1]}, \hat{B}')$.

Theorem 6.1 (Security of PC-PoK). In the ROM and under the 1-DLOG in $\mathbb{G}$, the PC-PoK proof system is perfectly complete, zero-knowledge, and knowledge-sound.

We prove this theorem in Appendix A.

6.2 System Model

Our model follows the standard threshold secret-sharing setting used in traceable/verifiable schemes, with three critical modifications to support accountability: (i) setup channels provide sender anonymity and origin authenticity via an AAC [4, 21], (ii) the participant set includes $t - 1$ dummy indices used for tracing, and (iii) tracing is executed by the dealer using dummy shares, after parties publicly release their verification keys upon trigger activation.

Reconstruction remains standard: any t valid shares recover the secret. The effective operational threshold is t out of $m = |I|$ real parties, as dummy shares are logically held by the dealer and utilized only during the forensic tracing phase.

Public Parameters. The construction is parameterized by $\text{pp} = (1^\lambda, n, t, \rho)$ and domain $\mathcal{G}(\lambda) = (\mathbb{G}, q, g, h_1, h_2)$ (an implicit global parameter fixed for security parameter λ and shared across all algorithms). ρ is a public random string (e.g., from a random beacon) used to derive a basis vector $\vec{\alpha} = (\alpha_1, \dots, \alpha_{t-1}) \in \mathbb{G}^{t-1}$ (e.g., via a hash-to-group procedure). The discrete logarithm relations among generators and basis elements are assumed unknown. For PC-PoK, the public parameter $\text{pp}^{\text{PC}} := (g, \vec{\alpha})$ is derived directly from pp .

Entities. The indexed universe $\mathcal{U} = \{\mathcal{P}_1, \dots, \mathcal{P}_n\}$ is partitioned into m real indices I and $t - 1$ dummy indices J . We enforce indistinguishability: given transcript T and pp , determining whether index $\ell \in [n]$ belongs to I or J is computationally hard. Dummy indices are logical participants whose network credentials are controlled by the dealer.

In addition to these participants, the system defines specific functional roles:

- *Combiner*: Aggregates shares to reconstruct the secret; this role can be assigned to any entity.
- *Trigger Authority*: Validates leakage evidence to activate tracing. In TSS-PV, this role is fixed as the Dealer.
- *Tracer*: Executes the tracing algorithm. This role is also fixed as the Dealer in TSS-PV.
- *Judge*: Verifies the tracing output (proof of guilt); can be any entity holding the verification key.
- *External Verifier*: Verifies the validity of the reconstructed secret without access to individual shares.

Channels (Sharing Phase). To preserve index indistinguishability, the sharing phase must be executed via an AAC [4, 21]. The AAC ensures origin authenticity while providing sender anonymity and unlinkability.

To prevent volume-based traffic analysis, the dealer must emit exactly n indistinguishable message flows via the AAC. For example, flows for $j \in J$ (dummies) can be routed through a mix-net to network endpoints controlled by the dealer, simulating legitimate delivery latency and volume. This ensures a network observer cannot distinguish the set of real parties I by counting outgoing packets.

Phases. The system proceeds as follows. The TSS-PV standard lifecycle includes Sharing and Reconstruction phases. When leakage evidence is confirmed, TSS-PV activates Tracing phase (or accountability mechanism) which includes Tracing Trigger and Tracing Execution sub-phases. The system overview is depicted in Fig. 9.

- *Standard Lifecycle*. Includes the Sharing and Reconstruction phases.
 - *Sharing*. The dealer $\mathcal{D}$ runs Share interactively with each party $\mathcal{P}_i$ for $i \in [n]$ over the AAC. Any entity can verify share distribution using VerSD.
 - *Reconstruction*. All parties submit shares to a combiner. It checks validity using VerSS and reconstructs S . External verifiers (who do not have access to shares) may optionally verify the output using VerRS.
- *Tracing (Accountability Mechanism)*. The tracing phase comprises two sub-phases:
 - *Trigger*: The dealer probes the reconstruction box R with subsets of dummy shares. If some subset size $t - f$ consistently yields a valid secret (via VerRS), the dealer determines the leakage size f and declares the trigger $\mathcal{T}$

Lagrange interpolation on the exponents. Note, in the reconstruction phase, the combiner runs VerSS on shares before running Rec.

```

Rec(SH)  $\rightarrow S/\perp$  :
1: if |SH| <  $\tau$  then return  $\perp$ ;
2: parse SH =  $\{sh_i\}_{i \in [t]}$ ;
3:  $\forall i \in [t]$  : parse  $sh_i = (x_i, p_i)$ ;
4: return  $S \leftarrow \prod_{j=1}^t p_j^{\Delta_j}$ , where  $\Delta_j \leftarrow \prod_{i=1, i \neq j}^t \frac{x_i}{x_i - x_j}$ ;

```

Figure 11: The reconstruction algorithm Rec of TSS-PV.

Share Distribution Verification. The public algorithm VerSD (Figure 12) enables external parties to confirm that the dealer used a single global $\omega = (r, s)$ across all n indices: the consistency check (Line 2) enforces one commitment, and the per-index NIZK check (Line 3) ensures each p_i was formed under that same commitment.

Share Submission Verification. Given a submitted share $sh_i = (x_i, p'_i)$ and the public transcript T , algorithm VerSS (Figure 12) checks that only the legitimate holder of evaluation point x_i could have produced this share: it recomputes $\tilde{f}_i$ from x_i (Line 4) and verifies that p'_i matches the dealer's committed value and passes the NIZK (Line 5).

Secret Verification Algorithm. The public algorithm VerRS (Figure 12) lets any party verify a candidate secret S' against the dealer's commitment without access to any shares: if S' is the secret, the global proof ψ_1 (as ψ_i is identical for all $i \in [n]$) passes the verification as $S' = g^s \wedge cs_1 = h_s^s$ for some witness s .

Remark 6.2 (Verification Algorithms Relationship). Algorithm VerSS re-runs the NIZK verification on index i , even though VerSD also verifies π_i . This redundancy is intentional: VerSS is designed to be sound even for verifiers who do not execute VerSD beforehand. Similarly, both VerSS and VerRS re-check global consistency (Line 2), ensuring each algorithm is self-contained. In settings where $\text{VerSD}(T) = 1$ is already guaranteed, the consistency check and the call to NIZK.Vr inside VerSS are logically implied by the global check together with the pre-share value- and share-binding tests ($\tilde{f}_i = f_i$ and $p'_i = p_i$), and may be omitted as an implementation-level optimization. Our security definitions and proofs, however, treat each verification algorithm as a standalone predicate that does not assume prior execution of the others.

Tracing Algorithm. Given tracing keys TK and the public transcript T , the tracer identifies which parties have their shares embedded in the reconstruction box R . The key insight is that a query $R(\text{DSH} \cup \{sh_i\})$ succeeds (i.e., $\text{VerRS} = 1$) if and only if sh_i is not in the box's internal leakage set E ; by Def. 5.8, queries containing a leaked share fail. The tracer thus tests each real index and removes those whose queries succeed, leaving only indices whose shares are embedded in R .

```

VerSD(T)  $\rightarrow \{0, 1\}$ :
1: parse  $T = \{(f_i, \tilde{V}_i, \pi_i^{\text{PC}}, p_i, \pi_i, \psi_i, C_i)\}_{i \in [n]}$ , where
    $C_i = (cr_i, cs_i) \forall i \in [n]$ ;
2: if  $\exists i, j \in [n] : \psi_i \neq \psi_j \vee C_i \neq C_j$  then return 0;
3: if  $\exists i \in [n]$  :
    $\text{NIZK.Vr}((p_i, cr_i, cs_i), ((f_i, g), (h_1, \mathbf{1}_G), (\mathbf{1}_G, h_2)), \pi_i) \neq 1$ 
   then return 0;
4: return 1;
VerSS(shi, T)  $\rightarrow \{0, 1\}$ :
1: parse  $T = \{(f_i, \tilde{V}_i, \pi_i^{\text{PC}}, p_i, \pi_i, \psi_i, C_i)\}_{i \in [n]}$ , where
    $C_i = (cr_i, cs_i) \forall i \in [n]$ ;
2: parse  $sh_i = (x_i, p'_i)$ ;
3: if  $\exists i, j \in [n] : \psi_i \neq \psi_j \vee C_i \neq C_j$  then return 0;
4:  $\tilde{f}_i \leftarrow \prod_{j=1}^{t-1} \alpha_j^{x_j}$ ;
5: if  $(\tilde{f}_i \neq f_i) \vee (p'_i \neq p_i)$ 
    $\vee (\text{NIZK.Vr}((p'_i, cr_i, cs_i), ((\tilde{f}_i, g), (h_1, \mathbf{1}_G), (\mathbf{1}_G, h_2)), \pi_i) \neq 1)$ 
   then return 0;
6: return 1;
VerRS(S', T)  $\rightarrow \{0, 1\}$ :
1: parse  $T = \{(f_i, \tilde{V}_i, \pi_i^{\text{PC}}, p_i, \pi_i, \psi_i, C_i)\}_{i \in [n]}$ , where
    $C_i = (cr_i, cs_i) \forall i \in [n]$ ;
2: if  $\exists i, j \in [n] : \psi_i \neq \psi_j \vee C_i \neq C_j$  then return 0;
3: if  $\text{NIZK.Vr}((S', cs_1), (g, h_2), \psi_1) \neq 1$  then return 0;
4: return 1;

```

Figure 12: The public verification algorithms. VerSD does not verify π_i^{PC} as π_i^{PC} attests party honesty which VerSS captures.

The output consists of the accused set $\mathcal{C}'$ together with evidence π_{trace} containing the corresponding shares. The dummy set size $t - f - 1$ is chosen so that DSH always falls short of threshold alone, forcing R to depend on the tested share sh_i to reconstruct.

Tracing Verification Algorithm. We adopt a replay-based design so that any auditor can independently exercise the reconstruction box without trusting the tracer's logs. Given $(VK, T, \mathcal{C}', \pi_{\text{trace}})$ and black-box access to R , the judge validates provenance (Lines 3–6) then replays R on each accused index (Lines 7–8): a successful query proves that index innocent, so any such result causes the entire accusation set $\mathcal{C}'$ to be rejected, binding the accusation to the observable behavior of R under publicly verifiable inputs.

Note, in our construction, the information required to trace a share is identical to the information required to verify it; hence tk_i and vk_i contain the same payload (the cleartext share). However, we maintain the distinction in the syntax to adhere to the generic TSS framework [10], supporting future instantiations where tracing and verification capabilities may rely on distinct cryptographic material.

$\text{Trace}^R(\text{TK}, T) \rightarrow (C', \pi_{\text{trace}})$:

- 1: **parse** $\text{TK} = \{(\text{ptk}_i, \text{dtk}_i)\}_{i \in [n]}$;
- 2: $I := \{i \in [n] : \text{ptk}_i \neq \perp\}$, $J := \{i \in [n] : \text{dtk}_i \neq \perp\}$;
- 3: $\forall i \in I$: **parse** $\text{ptk}_i = \text{sh}_i$, $\forall j \in J$: **parse** $\text{dtk}_j = \text{sh}_j$;
- 4: $\text{DSH} \leftarrow \{ \text{sh}_j \}_{j \in J}$, $|\text{DSH}| = t - f - 1$;
- 5: $C' := I$;
- 6: **for** $i \in I$:
 - (a) $S_i \leftarrow R(\text{DSH} \cup \{\text{sh}_i\})$;
 - (b) **if** $\text{VerRS}(S_i, T) = 1$ **then set** $C' := C' \setminus \{i\}$;
- 7: **set** $\pi_{\text{trace}} := \{\text{sh}_i\}_{i \in C'}$;
- 8: **return** (C', π_{trace}) ;

$\text{TrVer}^R(\text{VK}, T, C', \pi_{\text{trace}}) \rightarrow \{0, 1\}$:

- 1: **parse** $T = \{(f_i, \vec{V}_i, \pi_i^{\text{PC}}, p_i, \pi_i, \psi_i, C_i)\}_{i \in [n]}$, where $C_i = (c_{r_i}, c_{s_i}) \forall i \in [n]$;
- 2: **if** $\exists i, j \in [n] : \psi_i \neq \psi_j \vee C_i \neq C_j$ **then return** 0;
- 3: **parse** $\text{VK} = \{(\text{pvk}_i, \text{dvk}_i)\}_{i \in [n]}$;
- 4: $I := \{i \in [n] : \text{pvk}_i \neq \perp\}$, $J := \{i \in [n] : \text{dvk}_i \neq \perp\}$;
- 5: $\forall i \in I$: **parse** $\text{pvk}_i = \text{sh}_i$, $\forall j \in J$: **parse** $\text{dvk}_j = \text{sh}_j$;
- 6: **if** $(C' \not\subseteq I) \vee (\pi_{\text{trace}} \neq \{\text{sh}_i\}_{i \in C'}) \vee (\text{VerSD}(T) \neq 1) \vee (\exists i \in I : \text{VerSS}(\text{sh}_i, T) \neq 1) \vee (\exists j \in J : \text{VerSS}(\text{sh}_j, T) \neq 1)$ **then return** 0;
- 7: $\text{DSH} \leftarrow \{ \text{sh}_j \}_{j \in J}$, $|\text{DSH}| = t - f - 1$;
- 8: **if** $\exists i \in C' : (S' \leftarrow R(\text{DSH} \cup \{\text{sh}_i\})) \wedge (\text{VerRS}(S', T) = 1)$ **then return** 0;
- 9: **return** 1;

Figure 13: The tracing Trace^R and tracing verification algorithms TrVer^R . For simplicity, we assume the tracer and the judge have the coalition size f , which is determined in tracing trigger sub-phase.

6.4 Security analysis

In this section, we show that our TSS-PV satisfies security notions defined in Section 5. We prove the following theorems in Appendix B.

Theorem 6.2 (Perfect Correctness). Assume that the underlying NIZK proof systems are complete. Then our TSS-PV scheme satisfies perfect correctness.

Theorem 6.3 (Share Distribution Verifiability). Assume that the soundness of NIZK_π and share-submission verifiability (Def. 5.3) both hold. Then our TSS-PV scheme satisfies the verifiable distribution for the security parameter λ .

Theorem 6.4 (Share Submission Verifiability). Assume that the 1-DLOG assumption holds. Then our TSS-PV scheme satisfies verifiable submission for the security parameter λ .

Theorem 6.5 (Secret Reconstruction Verifiability). Assume that the soundness of NIZK_ψ holds. Then our TSS-PV scheme satisfies verifiable secret reconstruction for the security parameter λ .

Theorem 6.6 (Dealer Secrecy). Assume that the DDH assumption holds in $\mathbb{G}$, the underlying NIZK proof systems are zero-knowledge in the ROM, and the PC-PoK proof system is knowledge-sound in the ROM. Then our TSS-PV scheme satisfies dealer secrecy for the security parameter λ .

Theorem 6.7 (Party Secrecy). Assume that the $(t-1)$ -DLOG assumption holds in $\mathbb{G}$ and the PC-PoK proof system is zero-knowledge in the ROM. Then our TSS-PV scheme satisfies party secrecy for the security parameter λ .

Theorem 6.8 (Traceability). Assume that the correctness of our scheme holds, and the reconstruction box R is verifiability-aware and perfect. Then our TSS-PV scheme satisfies traceability for the security parameter λ .

Theorem 6.9 (Non-imputability). Assume that the party secrecy holds, all three verifiability properties (VerSD , VerSS , and VerRS) hold, and the reconstruction box R is verifiability-aware and perfect. Then our TSS-PV scheme satisfies non-imputability for the security parameter λ .

7 Performance

This section evaluates TSS-PV along three axes.

- (Q1) Do measured costs match the analytic predictions?
- (Q2) Where is the bottleneck, and how do costs scale with the number of parties?
- (Q3) Are the rare-event accountability costs practical at realistic party counts?

We first present closed-form cost expressions (Section 7.1), then validate them against a prototype implementation (Section 7.2).

7.1 Analytic Costs

We count group scalar multiplications (i.e., exponentiations / EC scalar mults) in $\mathbb{G}$, which dominate running time; group additions, hashing, and field operations are lower order unless stated. We analyze the construction in Section 6 for threshold t , m real parties, and $n = m + (t-1)$ total sharing instances, writing $f < t$ for the colluding coalition size from the traceability model.

Table 2 presents closed-form costs and their asymptotics. We separate tracer and judge work into caller-work (denoted as $\text{Trace}[\text{pub}]$ and $\text{TrVer}[\text{pub}]$, respectively) and reconstruction-box internal work (denoted as $\text{Trace}[\text{R}]$ and $\text{TrVer}[\text{R}]$, respectively) for two reasons: (i) the reconstruction box R may be deployed as a hardware or isolated software module, and its cost should be isolated; and (ii) the split mirrors the algorithms in Section 6 and aligns with our instrumented counters.

Table 2: Analytic costs for all phases of TSS-PV. Asymptotic bounds for Trace[R] and TrVer[R] assume $f = \Theta(t)$.

Phase	Scalar Mults in $\mathbb{G}$	Asymptotic
Share $\mathcal{P}_i$ (per party)	$5(t-1)$	$O(t)$
Share: $\mathcal{D}$ (total)	$5nt + m + 5$	$O(nt)$
Rec (on t shares)	t	$O(t)$
VerSD	$7n$	$O(n)$
VerSS (per submission)	$t+6$	$O(t)$
VerRS (per call)	4	$O(1)$
Trace[R]	$m(t+6)(t-f) + (m-f)t$	$O(mt^2)$
Trace[pub]	$4(m-f)$	$O(m)$
TrVer[R]	$f(t+6)(t-f)$	$O(ft^2)$
TrVer[pub]	$n(t+13)$	$O(nt)$
Transcript size	$(t+1)n + 2$ group elem.	$O(nt)$

7.2 Experimental Evaluation

We validate the analytic costs against a prototype implementation, then examine the scaling behavior and cost distribution across four configurations of increasing size.

7.2.1 Implementation and Setup

We describe the prototype, test configurations, and reconstruction-box instantiation used in all experiments.

Prototype and environment. We implemented TSS-PV over Curve25519 in Python with a Rust FFI backend (curve25519-dalek v4.1.3, AVX2). Experiments ran on an Intel Core i7-8700K at 4.4 GHz (6C/12T) with 32 GB DDR4-2400, Ubuntu 22.04 (kernel 5.15) under WSL2, with the process pinned to one core. We use four test configurations:

Configurations	(m, t, f)	$n = m + (t-1)$
C ₁	(32, 17, 11)	48
C ₂	(64, 33, 22)	96
C ₃	(128, 65, 43)	192
C ₄	(256, 129, 85)	384

We perform one warm-up and then ten measured trials, reporting medians. Wall-clock time is measured with `time.perf_counter()`, scalar multiplications are counted by instrumented wrappers, and peak RSS via `getrusage`.

Reconstruction box. The box R embeds f real shares, validates each submitted share via VerSS, enforces per-index uniqueness in x , and returns Rec on the first t unique valid shares (else $\perp$). Thus one R query incurs $(t+6)(t-f)$ scalar mults for validation and an additional t when reconstruction occurs. The reconstruction box R is implemented using the same codebase and runs on the same hardware as the caller-side code.

Artifact. An anonymized repository is available at <https://anonymous.4open.science/r/TSS-PV-07D6/>.

7.2.2 Results

Table 3 reports medians for latency (ms) and total scalar multiplications across all configurations; peak RSS remains below 28 MiB throughout. Figure 14 plots the public-phase and tracing latencies.

Analytic validation (Q1). Measured scalar-mult counts match the closed-form costs in Table 2 exactly: $5(t-1)$ per party for PC-PoK proof generation; $5nt + m + 5$ for the dealer; $7n$ for VerSD; $t+6$ per submission for VerSS; t for Rec; 4 for VerRS; and the stated forms for all tracing phases. Latency scales linearly with these counts on our platform.

Cost breakdown and scaling (Q2). In the standard lifecycle, the dealer dominates ($\approx 91\%$ of latency at C₄); public verification remains lightweight (VerSD: 108 ms at $n=384$; VerRS: sub-millisecond throughout). As m doubles from C₁ to C₄, dealer latency grows by $\approx 4\times$ per step (consistent with $O(nt)$ since both n and t roughly double), while per-party sharing grows by $\approx 2\times$ (consistent with $O(t)$). The dealer’s work is embarrassingly parallel across the n instances; a multi-threaded implementation would reduce wall-clock time proportionally.

Accountability costs (Q3). Trace[R] accounts for $\approx 97\%$ of total tracing cost at C₄ and exhibits the expected quadratic growth: from 223 ms at C₁ to 78.4 s at C₄ ($\approx 352\times$ as (m, t) quadruples), while caller-side Trace[pub] grows by only $8\times$. Tracing cannot be spam-triggered: no queries to R are issued unless $\mathcal{T}$ is satisfied (Def. 5.9). For moderate deployments (C₂, $m=64$), the standard lifecycle completes in under 700 ms and the full trace-and-verify cycle in under 2.1 s.

Comparability to prior work. No prior scheme simultaneously provides traceability against verifiability-aware black-box leakage and public verifiability, so a direct end-to-end comparison is impossible. Table 4 contrasts TSS-PV with the lattice-based PVSS of Gentry *et al.* [24] on group-exponentiation cost, omitting field arithmetic, LWE operations, and communication. By avoiding share encryption and heavyweight bulletproofs, our construction trades their $O(\lambda+n)$ sharing cost for $O(nt)$, while achieving λ -independent, $O(n)$ distribution verification. The trade-off favors TSS-PV when $t \ll \lambda$, which is the common regime for institutional secret sharing.

8 Discussion

We discuss the trade-offs inherent to our approach, the resulting operational constraints and the future work.

Box Persistence. Although related works [10, 28] assume the reconstruction box to be an oracle without explicit justification, we explicitly discuss its practical grounding. Our traceability analysis models R as a memoryless oracle, assuming the tracer can reset its state between queries. Our construction’s indistinguishable dummy shares prevent selective query

Table 3: Performance of TSS-PV. Each cell shows latency (ms) and scalar-mult count. [†]Share $\mathcal{P}_i$ is per-party measurement. ^{††}Share $\mathcal{D}$ is n -instance measurement. ^{†††}VerSS is measured for t submissions.

Function	C ₁ ms / #mult	C ₂ ms / #mult	C ₃ ms / #mult	C ₄ ms / #mult
<i>Standard Lifecycle</i>				
Share $\mathcal{P}_i^{\dagger}$	3.1 / 80	6.0 / 160	11.8 / 320	24.5 / 640
Share $\mathcal{D}^{\dagger\dagger}$	159.9 / 4117	604.2 / 15909	2424.9 / 62533	9714.4 / 247941
VerSD	14.0 / 336	27.1 / 672	52.7 / 1344	107.6 / 2688
VerSS ^{†††}	17.0 / 391	56.2 / 1287	210.2 / 4615	777.8 / 17415
Rec	1.4 / 17	3.8 / 33	11.7 / 65	42.4 / 129
<i>Accountability Mechanism</i>				
Trace[R]	222.7 / 4773	1422.3 / 28842	10048.0 / 205461	78365.7 / 1542699
Trace[pub]	3.2 / 84	6.6 / 168	13.6 / 340	25.8 / 684
TrVer[R]	66.9 / 1518	419.1 / 9438	2988.2 / 67166	23069.0 / 504900
TrVer[pub]	62.3 / 1440	201.0 / 4416	695.0 / 14976	2537.4 / 54528

Table 4: Asymptotic group-exponentiation cost (Standard Lifecycle) comparison.

Scheme	Share	Dist. Verif.	Subm. Verif.	Rec	Rec. Verif.
[24]	$O(\lambda+n)^a$	$O(\lambda+n)$	— ^b	$O(1)^c$	—
TSS-PV	$O(nt)^d$	$O(n)$	$O(t)$	$O(t)$	$O(1)$

^aTSS-PV counts are λ -independent; [24] verifiers additionally pay $O(\lambda^2 + \lambda n)$ field mults. ^bImplicit in global Bulletproof. ^cDominated by LWE; group work $O(1)$. ^dDealer cost; parties also pay $O(t)$.

filtering, any evasion degrades service for all users, and the resulting economic friction (significant counterparty risk when transmitting shares to anonymous endpoints [1, 29]) incentivizes adversaries to distribute local artifacts (e.g., binaries or dongles) subject to forensic capture. Once captured, achieving truly tamper-proof (uncloable and unresettable) hardware remains an open challenge despite TEE anti-rollback features and PUF-based key storage, as demonstrated by recent attacks [16, 34, 36].

Tracing Liveness. Tracing requires key release from the dealer and all n parties, which is stricter than the t -of- n reconstruction threshold. However, tracing is invoked only upon leakage evidence; during normal operation, reconstruction proceeds with any t honest parties. While we cannot cryptographically compel participation, application-layer policies can treat non-cooperation during a forensic audit as a policy violation, narrowing the set of suspects even without cryptographic attribution.

Future Work. Several directions remain to strengthen guarantees and bridge theory with deployment. First, real-world adversaries may construct boxes that deviate from the perfect model (e.g., probabilistic acceptance, stateful evasion); future work could formalize ϵ -good boxes and achieve traceability

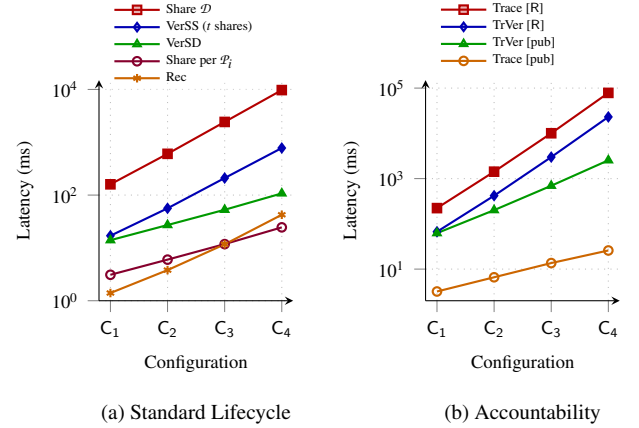


Figure 14: Performance of TSS-PV across configurations C₁–C₄. (a) Standard lifecycle: dealer costs dominate due to PC-PoK proof verification, while public verification remains lightweight. (b) Accountability mechanism: reconstruction-box queries dominate, while caller-side checks remain modest.

via query amplification, repeated testing, or weakened trust assumptions such as honest-dealer non-immutability or trusted setup. Second, blockchain-based time-locked commitments with staked collateral could penalize non-participating parties, transforming forensic obstruction into economic loss. Third, efficient realizations of Anonymous Authenticated Channels over mix networks or TEE-based relays require characterizing the minimal anonymity guarantees for index indistinguishability. Finally, integrating Distributed Key Generation would eliminate the dealer as a single point of failure.

References

- [1] Christian Badertscher, Juan Garay, Ueli Maurer, Daniel Tschudi, and Vassilis Zikas. But why does it work? a rational protocol design treatment of bitcoin. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 34–65. Springer, 2018.
- [2] Karim Baghery, Ehsan Ebrahimi, Omid Mirzamohammadi, and Mahdi Sedaghat. Traceable verifiable secret sharing and applications. *Cryptology ePrint Archive*, 2025.
- [3] Ali Bagherzandi, Jung-Hee Cheon, and Stanislaw Jarecki. Multisignatures secure under the discrete logarithm assumption and a generalized forking lemma. In *Proceedings of the 15th ACM conference on Computer and communications security*, pages 449–458, 2008.
- [4] Fabio Banfi and Ueli Maurer. Anonymous authenticated communication. In *International Conference on Security and Cryptography for Networks*, pages 289–312. Springer, 2022.
- [5] Mihir Bellare and Gregory Neven. Multi-signatures in the plain public-key model and a general forking lemma. In *Proceedings of the 13th ACM conference on Computer and communications security*, pages 390–399, 2006.
- [6] Allison Bishop, Matthew Green, Yuval Ishai, Abhishek Jain, and Paul Lou. Fully anonymous secret sharing. In *Annual International Cryptology Conference*, pages 356–389. Springer, 2025.
- [7] George Robert Blakley. Safeguarding cryptographic keys. In *Managing requirements knowledge, international workshop on*, pages 313–313. IEEE Computer Society, 1979.
- [8] Carlo Blundo and Douglas R Stinson. Anonymous secret sharing schemes. *Discrete Applied Mathematics*, 77(1):13–28, 1997.
- [9] Dan Boneh. The decision diffie-hellman problem. In *International algorithmic number theory symposium*, pages 48–63. Springer, 1998.
- [10] Dan Boneh, Aditi Partap, and Lior Rotem. Traceable secret sharing: Strong security and efficient constructions. In *Annual International Cryptology Conference*, pages 221–256. Springer, 2024.
- [11] Jan Camenisch and Markus Stadler. Proof systems for general statements about discrete logarithms. *Technical Report/ETH Zurich, Department of Computer Science*, 260, 1997.
- [12] Ignacio Cascudo and Bernardo David. Publicly verifiable secret sharing over class groups and applications to dkg and yoso. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 216–248. Springer, 2024.
- [13] Hervé Chabanne, Duong Hieu Phan, and David Pointcheval. Public traceability in traitor tracing schemes. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 542–558. Springer, 2005.
- [14] David Chaum and Torben Pryds Pedersen. Wallet databases with observers. In *Annual international cryptology conference*, pages 89–105. Springer, 1992.
- [15] Yilei Chen, Vinod Vaikuntanathan, Brent Waters, Hoeteck Wee, and Daniel Wichs. Traitor-tracing from lwe made simple and attribute-based. In *Theory of Cryptography Conference*, pages 341–369. Springer, 2018.
- [16] J Chuang, A Seto, N Berrios, S van Schoch, C Garman, and D Genkin. Tee. fail: Breaking trusted execution environments via ddr5 memory bus interposition. In *2026 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, pages 1894–1912, 2026.
- [17] Xuan Thanh Do, Duong Hieu Phan, and David Pointcheval. Traceable inner product functional encryption. In *Cryptographers’ Track at the RSA Conference*, pages 564–585. Springer, 2020.
- [18] Oriol Farràs and Miquel Guiot. Traceable secret sharing schemes for general access structures. *Cryptology ePrint Archive*, 2025.
- [19] Paul Feldman. A practical scheme for non-interactive verifiable secret sharing. In *28th Annual Symposium on Foundations of Computer Science (sfcs 1987)*, pages 427–438. IEEE, 1987.
- [20] Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In *Conference on the theory and application of cryptographic techniques*, pages 186–194. Springer, 1986.
- [21] Kyle Fredrickson, Ioannis Demertzis, James P Hughes, and Darrell DE Long. Sparta: Practical anonymity with long-term resistance to traffic analysis. In *2025 IEEE Symposium on Security and Privacy (SP)*, pages 1457–1473. IEEE, 2025.
- [22] Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In *Advances in Cryptology—CRYPTO 2018: 38th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 19–23, 2018, Proceedings, Part II* 38, pages 33–62. Springer, 2018.

- [23] Sanjam Garg, Abishek Kumarasubramanian, Amit Sahai, and Brent Waters. Building efficient fully collusion-resilient traitor tracing and revocation schemes. In *Proceedings of the 17th ACM conference on computer and communications security*, pages 121–130, 2010.
- [24] Craig Gentry, Shai Halevi, and Vadim Lyubashevsky. Practical non-interactive publicly verifiable secret sharing with thousands of parties. In *Annual international conference on the theory and applications of cryptographic techniques*, pages 458–487. Springer, 2022.
- [25] Junqing Gong, Ji Luo, and Hoeteck Wee. Traitor tracing with $n^{1/3}$ -size ciphertexts and $o(1)$ -size keys from k -lin. In *Annual International Conference on the Theory and Applications of Cryptographic Techniques*, pages 637–668. Springer, 2023.
- [26] Rishab Goyal, Venkata Koppula, and Brent Waters. Collusion resistant traitor tracing from learning with errors. In *Proceedings of the 50th annual ACM SIGACT symposium on theory of computing*, pages 660–670, 2018.
- [27] Vipul Goyal, Abhishek Jain, and Aditi Partap. Traceable secret sharing revisited. *Cryptology ePrint Archive*, 2025.
- [28] Vipul Goyal, Yifan Song, and Akshayaram Srinivasan. Traceable secret sharing and applications. In *Advances in Cryptology—CRYPTO 2021: 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16–20, 2021, Proceedings, Part III 41*, pages 718–747. Springer, 2021.
- [29] Joseph Y Halpern and Rafael Pass. Game theory with translucent players. *International Journal of Game Theory*, 47(3):949–976, 2018.
- [30] Wataru Kishimoto, Koji Okada, Kaoru Kurosawa, and Wakaha Ogata. On the bound for anonymous secret sharing schemes. *Discrete Applied Mathematics*, 121(1-3):193–202, 2002.
- [31] Pham Nhat Minh, Khoa Nguyen, Willy Susilo, and Khuong Nguyen-An. Publicly verifiable secret sharing: Generic constructions and lattice-based instantiations in the standard model. *arXiv preprint arXiv:2504.14381*, 2025.
- [32] Torben Pryds Pedersen. Non-interactive and information-theoretic secure verifiable secret sharing. In *Annual international cryptology conference*, pages 129–140. Springer, 1991.
- [33] Claus-Peter Schnorr. Efficient identification and signatures for smart cards. In *Conference on the Theory and Application of Cryptology*, pages 239–252. Springer, 1989.
- [34] Alex Seto, Oytun Kaday Duran, Samy Amer, Jalen Chuang, Stephan van Schaik, Daniel Genkin, and Christina Garman. Wiretap: Breaking server sgx via dram bus interposition. In *Proceedings of the 2025 ACM SIGSAC Conference on Computer and Communications Security*, pages 708–722, 2025.
- [35] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [36] Supraja Sridhara, Andrin Bertschi, Benedict Schlüter, and Shweta Shinde. Sigy: Breaking intel sgx enclaves with malicious exceptions & signals. In *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, pages 1643–1658, 2025.
- [37] Markus Stadler. Publicly verifiable secret sharing. In *International Conference on the Theory and Applications of Cryptographic Techniques*, pages 190–199. Springer, 1996.
- [38] Douglas R Stinson and Scott A Vanstone. A combinatorial approach to threshold schemes. *SIAM Journal on Discrete Mathematics*, 1(2):230–236, 1988.
- [39] Mark Zhandry. New techniques for traitor tracing: Size and more from pairings. In *Annual international cryptology conference*, pages 652–682. Springer, 2020.

A PC-PoK Security Proof

We proof Theorem 6.1 as follows.

Proof. Completeness follows since, for all $k \in [t-2]$, $A'_k = V_k^{z_1} V_{k+1}^{-c} = V_k^{r_1+cx} (V_k^x)^{-c} = V_k^{r_1} = A_k$ and for all $k \in [t-1]$, $B'_k = g^{z_k} V_k^{-c} = g^{r_k+cx^k} (g^{x^k})^{-c} = g^{r_k} = B_k$. The proof for $\hat{B}'$ is analogous.

For zero-knowledge, the simulator $\mathcal{S}$ samples random c and $\vec{z}$, computes commitments backwards via the verification equations, and programs $H(\text{pp}^{\text{PC}}, f, \vec{V}, \{A_k\}, \{B_k\}, \hat{B}) = c$. The transcript is identically distributed since $(c, \vec{z})$ are uniform in both cases.

For knowledge soundness, let x be the solution of 1-DLOG problem. By the Forking Lemma [5], two valid $(c, \vec{z})$ and $(c', \vec{z}')$, where $c \neq c'$ for the same commitments, can be obtained with non-negligible probability. This implies the existence of extracted values $x^k = (z_k - z'_k)/(c - c')$ for all $k \in [t-1]$. Hence, the solution x can be easily derived by $x = x^k \cdot (x^{k-1})^{-1}$. $\square$

B TSS-PV Security Proof

This section proves security Theorems in Section 6.4.

B.1 Correctness

We prove Theorem 6.2 as follows.

Proof. In the experiment $\text{ExpCor}_{\text{TSS-PV}}^{n,t}$, the honest execution of Share yields public parameters $\text{pp} = (1^\lambda, n, t, \rho)$, transcripts $\mathbf{T} = \{T_i\}_{i \in [n]}$, and shares $\Sigma = \{\text{sh}_i\}_{i \in [n]}$.

The dealer samples $(r, s) \leftarrow \mathbb{Z}_q^2$, sets $S \leftarrow g^s$, and for each $i \in [n]$ computes:

$$f_i = \prod_{j=1}^{t-1} \alpha_j^{x_i^j} \quad \text{and} \quad p_i = f_i^r \cdot g^s.$$

Let $\alpha_j = g^{a_j}$ where $a_j \in \mathbb{Z}_q$ is the discrete logarithm of the parameter α_j . Substituting this into the expression for p_i :

$$p_i = \left(\prod_{j=1}^{t-1} g^{a_j x_i^j} \right)^r \cdot g^s = g^{\sum_{j=1}^{t-1} (ra_j) x_i^j + s}.$$

This form reveals that $p_i = g^{\tilde{q}(x_i)}$, where $\tilde{q}(X) = s + \sum_{j=1}^{t-1} (ra_j) X^j$ is a polynomial of degree at most $t-1$ over $\mathbb{Z}_q$ with constant term $\tilde{q}(0) = s$.

Let $\mathcal{V} \subseteq [n]$ be any subset of valid shares with $|\mathcal{V}| = t$, and let $\text{SH} = \{(x_i, p_i)\}_{i \in \mathcal{V}}$. The protocol explicitly enforces that all f_i are unique and $f_i \neq 1_{\mathbb{G}}$. Therefore, the indices $\{x_i\}_{i \in \mathcal{V}}$ are pairwise distinct and non-zero, ensuring Lagrange interpolation applies. Define the Lagrange basis polynomials at zero as $\Delta_i = \prod_{k \in \mathcal{V}, k \neq i} \frac{x_k}{x_k - x_i}$. By the properties of polynomial interpolation, $\sum_{i \in \mathcal{V}} \Delta_i \tilde{q}(x_i) = \tilde{q}(0) = s$. The reconstruction algorithm $\text{Rec}(\text{SH})$ computes:

$$\prod_{i \in \mathcal{V}} p_i^{\Delta_i} = \prod_{i \in \mathcal{V}} g^{\Delta_i \tilde{q}(x_i)} = g^{\sum_{i \in \mathcal{V}} \Delta_i \tilde{q}(x_i)} = g^s = S.$$

Thus, Rec correctly recovers the secret.

For verification, each honest party generates π_i^{PC} using $\text{PC.Prf}(\text{pp}^{\text{PC}}, x_i)$. By the completeness of the PC-PoK, the dealer accepts. Similarly, the dealer generates π_i and ψ honestly. By the completeness of the underlying NIZK system, $\text{VerSD}(\mathbf{T})$, $\text{VerSS}(\text{sh}_i, \mathbf{T})$, and $\text{VerRS}(S, \mathbf{T})$ all return 1. $\square$

B.2 Verifiability

B.2.1 Share Distribution Verifiability

We prove share-distribution verifiability in Theorem 6.3 as follows.

Proof. Let $\mathcal{A}$ be an adversary of $\text{ExpVSD}_{\mathcal{A}, \text{TSS-PV}}$, acting as a malicious dealer. $\mathcal{A}$ directly outputs a transcript $\mathbf{T} = \{T_i\}_{i \in [n]}$, where $T_i = (f_i, \vec{v}_i, \pi_i^{\text{PC}}, p_i, \pi_i, \psi_i, C_i)$, shares $\Sigma = \{\text{sh}_i\}_{i \in [n]}$, and index sets $I_0, I_1 \subseteq [n]$ with $|I_0| = |I_1| = t$. Assume $\mathcal{A}$ wins. Then there exist index sets $I_0, I_1 \subseteq [n]$ with $|I_0| = |I_1| = t$, such that $\text{SH}_b = \{\text{sh}_i\}_{i \in I_b}$ satisfies $\text{VerSD}(\mathbf{T}) = 1$ and $S_0 \neq S_1$, where $S_b = \text{Rec}(\text{SH}_b)$. From $\text{VerSD}(\mathbf{T}) = 1$

and the soundness of the proofs $\{\pi_i\}$, there exist $r, s \in \mathbb{Z}_q$ such that, for every $i \in [n]$,

$$p_i = f_i^r \cdot g^s, \text{cr}_i = h_1^r, \text{and } \text{cs}_i = h_2^s,$$

and the commitments $C_i = (\text{cr}_i, \text{cs}_i)$ are identical across all indices; equivalently, the same global $\omega = (r, s)$ is used.

Let $\mathcal{V}(\mathbf{T}, \Sigma) = \{\text{sh}_i \in \Sigma : \text{VerSS}(\text{sh}_i, \mathbf{T}) = 1\}$, which has size at least t . Since $\text{SH}_0, \text{SH}_1 \subseteq \mathcal{V}(\mathbf{T}, \Sigma)$, every share used to reconstruct S_0 and S_1 is consistent with the same $\omega = (r, s)$. Hence, the reconstruction is uniquely determined by $S = g^s$, and

$$\text{Rec}(\text{SH}_0) = \text{Rec}(\text{SH}_1) = S,$$

contradicting $S_0 \neq S_1$, unless VerSS is violated or some proof π_i accepts a false statement, which occurs with probability $\epsilon_\pi^{\text{snd}}$. Consequently,

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{vsd}}(\lambda) \leq \text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{vss}}(\lambda) + n \cdot \epsilon_\pi^{\text{snd}},$$

which is negligible. $\square$

B.2.2 Share Submission Verifiability

We prove share-submission verifiability in Theorem 6.4 as follows.

Proof. In $\text{ExpVSS}_{\mathcal{A}, \text{TSS-PV}}$ with an honest dealer, the adversary $\mathcal{A}$ can succeed only in one of the following events: (i) W_0 : it outputs distinct valid submissions sh_{i^*} and sh'_{i^*} for the same index, or (ii) W_1 : it outputs t valid submissions that reconstruct to $S' \neq S$.

• **(Event W_0)** Assume $\mathcal{A}$ outputs two accepting shares $\text{sh}_{i^*} = (x, y)$ and $\text{sh}'_{i^*} = (x', y')$ for the same transcript $T_{i^*} = (f_{i^*}, \vec{v}_{i^*}, \pi_{i^*}^{\text{PC}}, p_{i^*}, \pi_{i^*}, \psi_{i^*}, C_{i^*})$. By the definition of VerSS ,

$$y = y' = p_{i^*} \quad \text{and} \quad \prod_{j=1}^{t-1} \alpha_j^{x_j} = \prod_{j=1}^{t-1} \alpha_j^{x'_j} = f_{i^*}.$$

To reduce to the 1-DLOG problem, let $(g, Z = g^z)$ be a 1-DLOG instance. A reduction algorithm $\mathcal{B}$ simulates $\rho = (\alpha_1, \dots, \alpha_{t-1})$ as

$$\alpha_j = Z^{s_j} g^{t_j}, \quad \text{where } s_j, t_j \leftarrow \mathbb{Z}_q \quad (j \in [t-1]).$$

$\mathcal{B}$ runs the dealer honestly otherwise. Then the exponents $\{e_j = s_j z + t_j\}$ are independently uniform in $\mathbb{Z}_q$, so ρ is identically distributed to the real setup. Let $d_j = x^j - x'^j$ (not all zero as $x \neq x'$). Then discrete logarithms for f_{i^*} in the equality yield

$$\sum_{j=1}^{t-1} (s_j z + t_j) d_j = 0.$$

Hence, $\mathcal{B}$ recovers the solution of the 1-DLOG problem by

$$z = - \sum_{j=1}^{t-1} t_j d_j \cdot \left(\sum_{j=1}^{t-1} s_j d_j \right)^{-1}.$$

Clearly, the denominator above becomes 0 with probability at most $1/q$ as $\{s_1, \dots, s_{t-1}\}$ are uniform and independent. Hence, $\Pr[W_0](1 - 1/q) \leq \epsilon_{1-dl}$, where ϵ_{1-dl} is the probability that the 1-DLOG assumption is broken.

• **(Event W_1)** Since the dealer is honest, the global proof ψ , all proofs $\{\pi_i\}$, and commitments $\{C_i = (cr_i, cs_i)\}$ are honestly generated. Verification of ψ together with identical commitments across indices ensures that an identical $\omega = (r, s)$ is used for every i . This implies that every accepted share embeds the common secret $S = g^s$. Hence, this case never happens; i.e., $\Pr[W_1] = 0$.

Overall,

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{vss}}(\lambda) = \Pr[W_0 \vee W_1] \leq \frac{\epsilon_{1-dl}}{1 - 1/q}.$$

□

B.2.3 Secret Reconstruction Verifiability

We prove share-reconstruction verifiability in Theorem 6.5 as follows.

Proof. In $\text{ExpVRS}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$, the adversary $\mathcal{A}$ outputs a transcript $T = \{T_i\}_{i \in [n]}$ and two distinct candidates S_0, S_1 with $S_0 \neq S_1$. Suppose $\mathcal{A}$ wins. Then $\text{VerRS}(S_0, T) = \text{VerRS}(S_1, T) = 1$. By the definition of VerRS , all global commitments and proofs are consistent; in particular, $C_i = (cr, cs)$ and $\psi_i = \psi$ do not depend on i . Moreover, acceptance for S_b means that, by soundness of the NIZK_ψ , there exists $s_b \in \mathbb{Z}_q$ such that

$$S_b = g^{s_b} \quad \text{and} \quad cs = h_2^{s_b} \quad \text{where} \quad b \in \{0, 1\}.$$

By the soundness of the underlying NIZK system, there exist witnesses s_0, s_1 satisfying the above equalities. Since $cs = h_2^{s_0} = h_2^{s_1}$, it follows that $s_0 = s_1$ and, therefore, $S_0 = g^{s_0} = g^{s_1} = S_1$. This contradicts $S_0 \neq S_1$.

Hence, a successful $\mathcal{A}$ would break the soundness for ψ , which occurs with probability at most $\epsilon_\psi^{\text{snd}}$. That is,

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{vrec}}(\lambda) \leq \epsilon_\psi^{\text{snd}},$$

which is negligible. □

B.3 Secrecy

B.3.1 Dealer Secrecy

We prove the dealer-secrecy property in Theorem 6.6 as follows.

Proof. Let $\mathcal{B}$ be a DDH distinguisher given $(g, A = g^a, B = g^b, Z)$, where $Z = g^{ab}$ or $Z \leftarrow \mathbb{G}$. $\mathcal{B}$ simulates $\text{ExpDSec}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$ for an adversary $\mathcal{A}$ as follows.

$\mathcal{B}$ sets $h_1 = g^e$ for $e \leftarrow \mathbb{Z}_q$ and $cr = A^e = h_1^a$, implicitly embedding the dealer randomness $r = a$. Next, $\mathcal{B}$ samples

$h_2 \leftarrow \mathbb{G}$ and $s \leftarrow \mathbb{Z}_q$, and sets $cs = h_2^s$ and $S = g^s$. To generate $\rho = \vec{\alpha} \in \mathbb{G}^{t-1}$, for all $j \in [t-1]$, $\mathcal{B}$ samples secret scalars $c_j, w_j \leftarrow \mathbb{Z}_q$ and sets

$$\alpha_j = B^{c_j} \cdot g^{w_j} = g^{bc_j + w_j}.$$

Note that c_j is information-theoretically hidden from $\mathcal{A}$ by the one-time pad w_j . $\mathcal{B}$ then sends pp to $\mathcal{A}$. Before the share and proof generation of Share , $\mathcal{A}$ outputs, in a single round, a vector of transcripts $\{(f_i, \tilde{V}_i, \pi_i^{\text{PC}})\}_{i \in C}$, where C is the index set of corrupted parties. $\mathcal{B}$ verifies all proofs and checks that $f_i \neq 1_{\mathbb{G}}$ for each i ; if any check fails, $\mathcal{B}$ aborts. $\mathcal{B}$ then rewinds $\mathcal{A}$ to extract the witness vector $\{x_i\}_{i \in C}$. This is possible in expected polynomial time, due to the generalized forking lemma [3]. The extraction succeeds except with probability ϵ_{fork} . For honest parties $i \in \{t, \dots, n\}$, $\mathcal{B}$ samples $x_i \leftarrow \mathbb{Z}_q \setminus \{0\}$ and generates valid PC-PoK proofs. While the share and proof generation for $i \in [n]$, using the known or extracted x_i , $\mathcal{B}$ computes the pre-share $f_i = \prod_{j=1}^{t-1} \alpha_j^{x_i^j}$ and the simulated share

$$p_i = \left(\prod_{j=1}^{t-1} (Z^{c_j} \cdot A^{w_j})^{x_i^j} \right) \cdot S.$$

The proofs π_i and ψ are simulated via the zero-knowledge simulator, without requiring knowledge of a . The global commitment $C = (cr, cs)$ and proof ψ are sent identically to all parties, satisfying the protocol's consistency requirement.

The simulated p_i is distributed depending on Z , as follows:

- **(Real: $Z = g^{ab}$)** Since $Z^{c_j} \cdot A^{w_j} = g^{abc_j + aw_j} = \alpha_j^{c_j}$, the equality $p_i = f_i^a \cdot S$ holds, which is identically distributed to a real execution with dealer randomness $r = a$.
- **(Random: $Z = g^{ab+z'}$)** Let $z' \neq 0$. Then the share becomes

$$p_i = f_i^a \cdot S \cdot g^{z' M_i}, \quad \text{where} \quad M_i = \sum_{j=1}^{t-1} c_j x_i^j.$$

Since ρ statistically hides $\vec{c} = (c_1, \dots, c_{t-1})$, $\mathcal{A}$'s choice of x_i is independent of $\vec{c}$. Moreover, the protocol enforces $x_i \neq 0$, so the vector $(x_i, x_i^2, \dots, x_i^{t-1})$ is non-zero. Hence, M_i is uniform which implies $\Pr[M_i = 0] = 1/q$. When $M_i \neq 0$, the term $g^{z' M_i}$ acts as a random mask of S , yielding p_i uniformly distributed and independent of S .

Finally, $\mathcal{A}$ outputs S^* ; $\mathcal{B}$ outputs 1 if $S^* = S$, and 0 otherwise. The simulation fails only if the following events occurs: (1) the extraction fails with probability ϵ_{fork} , (2) $M_i = 0$ for some party with probability at most n/q by the union bound, or (3) $\mathcal{A}$ distinguishes the simulated ZK proofs with probability ϵ_{zk} . Hence,

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{dsec}}(\lambda) \leq \epsilon_{\text{ddh}} + \epsilon_{\text{zk}} + \epsilon_{\text{fork}} + \frac{n}{q}.$$

□

B.3.2 Party Secrecy

We prove the party-secrecy in Theorem 6.7 as follows.

Proof. Let $\mathcal{B}$ be a reduction algorithm given a $(t-1)$ -DLOG instance $(g, Z_1, \dots, Z_{t-1})$ where $Z_j = g^{z^j}$ for a hidden $z \in \mathbb{Z}_q$. Since in $\mathbf{ExpPsec}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$, the adversary $\mathcal{A}$ chooses the parameters n and t , $\mathcal{B}$ first guesses a value $t' \leftarrow_{\$} t_{\max}$ (which is a polynomial bound in λ). If $t \neq t'$ $\mathcal{B}$ aborts. Otherwise, $\mathcal{B}$ continues the following simulation:

After obtaining the target index i^* from $\mathcal{A}$, $\mathcal{B}$ samples $s_1, \dots, s_{t-1} \leftarrow_{\$} \mathbb{Z}_q$ and sets $\alpha_j = g^{s_j}$ for $j \in [t-1]$. Then $\rho = \vec{\alpha} \in \mathbb{G}^{t-1}$ is published. As the honest party i^* , $\mathcal{B}$ executes Share against the malicious dealer controlled by $\mathcal{A}$, and sends

$$f_{i^*} = \prod_{j=1}^{t-1} Z_j^{s_j} = g^{\sum_{j=1}^{t-1} s_j z^j},$$

together with $\vec{V}_{i^*} = (Z_1, \dots, Z_{t-1})$. Since $\mathcal{B}$ does not know z , by zero-knowledge of PC-PoK (Theorem 6.1), it simulates $\pi_{i^*}^{\text{PC}}$ without the witness.

This simulation implicitly fixes the honest secret of i^* to $x_{i^*} = z$, while preserving the distribution of a real execution. If $\mathcal{A}$ finally outputs $\hat{\text{sh}}_{i^*}$ including $\hat{x}$, $\mathcal{B}$ checks $g^{\hat{x}} = Z_j$ for all $j \in [t-1]$. If all checks pass, it outputs $\hat{x}$ as the solution to the $(t-1)$ -DLOG problem. By the winning condition, a success of $\mathcal{A}$ implies $\hat{x} = x_{i^*} = z$. Thus, whenever $\mathcal{A}$ wins, $\mathcal{B}$ solves the problem. Consequently,

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{psec}}(\lambda) \leq \varepsilon_{(t-1)\text{-dl}} \cdot t_{\max} + \varepsilon_{\text{zk}},$$

where $\varepsilon_{(t-1)\text{-dl}}$ denotes the advantage against $(t-1)$ -DLOG and ε_{zk} the zero-knowledge distinguishing advantage. $\square$

B.4 Traceability

In this section, we prove the Theorem 6.8.

Proof. Let $\mathcal{A}$ be the adversary against $\mathbf{ExpTrace}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$. After receiving pp and outputs of honest Share executions, $\mathcal{A}$ adaptively outputs a coalition $C \subseteq [n]$, a reconstruction box R, and the party outputs $\{\text{op}_i\}_{i \in [n]}$, where $|C| = f$.

Let $\Sigma = \{\text{sh}_i\}_{i \in [n]}$ be the set of shares generated by Share, and let $\mathcal{V}(\text{T}, \Sigma) = \{\text{sh} \in \Sigma : \text{VerSS}(\text{sh}, \text{T}) = 1\}$. $\mathcal{A}$'s success requires that the trigger $\mathcal{T}(\text{T}, \Sigma, \text{R}; C)$ holds and tracing fails to identify. Consider $E = \{\text{sh}_i : i \in C\}$. By the definition of a verifiability-aware perfect box R (Definition 5.8), there exists a set $\text{SH} \subseteq \mathcal{V}(\text{T}, \Sigma)$ with $|\text{SH}| = t - f$ and $\text{SH} \cap E = \emptyset$, such that $\text{VerRS}(\text{R}(\text{SH}), \text{T}) = 1$.

Let TK and VK denote, respectively, the tracing key and verification key collections generated in the same Share executions. Using TK, the tracer Trace^{R} fixes a dummy set DSH of size $t - f - 1$. For each $i \in [n]$, consider the set $\text{DSH} \cup \{\text{sh}_i\}$. (i) if $i \in C$ (i.e., the index is indeed corrupted) then $\text{sh}_i \in E$ and thus $\text{DSH} \cup \{\text{sh}_i\}$ intersects E; the verification algorithm

VerRS does not accept $\text{R}(\text{DSH} \cup \{\text{sh}_i\})$. (ii) Otherwise, i.e., $\text{sh}_i \notin E$, $\text{DSH} \cup \{\text{sh}_i\}$ is a valid set disjoint from E. Therefore, R returns a secret S_i which VerRS accepts, and sets $C' = C' \setminus \{i\}$, where C' is the accusation set and is initialized by $C' := I$. Aggregating these outcomes over all i yields the exact coalition $C' = C$ and evidence π_{trace} produced by Trace^{R} .

Since the keys and transcripts were generated by the honest procedure, $\text{VerSD}(\text{T}) = 1$. Moreover, every share in $\pi_{\text{trace}} = \{\text{sh}_i\}_{i \in C}$ lies in $\mathcal{V}(\text{T}, \Sigma)$ and hence, passes VerSS . Finally, the accept/reject pattern attested in π_{trace} matches the behavior guaranteed by the verifiability-aware semantics of R together with VerRS . The public verifier can recompute these reconstructions using VK and T, thus accepting $\text{TrVer}^{\text{R}}(\text{VK}, \text{T}, C', \pi_{\text{trace}}) = 1$.

$\mathcal{A}$ can succeed only if a correctness failure occurs in one of VerSD , VerRS , or VerSS . Consequently,

$$\text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{trace}}(\lambda) \leq \varepsilon_{\text{cor}},$$

where ε_{cor} denotes the maximal correctness error of these verifiers, which equals 0 for our scheme by Theorem 6.2. $\square$

B.5 Non-imputability

We prove the non-imputability property in Theorem 6.9 as follows.

Proof. In $\mathbf{ExpNI}_{\mathcal{A}, \text{TSS-PV}}(\lambda)$, the adversary $\mathcal{A}$, given a public parameter pp, fixes a target index i^* and acts as the dealer. Let x_{i^*} denote the private input of the honest player indexed i^* . $\mathcal{A}$ runs Share with the honest party. This yields a transcript $T_{i^*} = (f_{i^*}, p_{i^*}, \pi_{i^*}, \psi_{i^*}, C_{i^*})$ associated with $\text{sh}_{i^*} = (x_{i^*}, p_{i^*})$. At the end, $\mathcal{A}$'s final output determines an accused set C, evidence π_{trace} , shares Σ , verification keys VK (which encode the shares) and transcripts T, and a reconstruction box R. The winning event is

$$W = (i^* \in C) \wedge (\text{TrVer}^{\text{R}}(\text{VK}, \text{T}, C, \pi_{\text{trace}}) = 1).$$

Let W_0 be the event that x_{i^*} is leaked to $\mathcal{A}$, and $W_1 = \neg W_0$.

• **(Event W_0)** By the party secrecy, $\mathcal{A}$ can obtain x_{i^*} except with at most $\varepsilon_{\text{psec}}$ (the probability of violating the party secrecy); that is, $\Pr[W_0] \leq \varepsilon_{\text{psec}}$.

• **(Event W_1)** Let $\mathcal{V}(\text{T}, \Sigma)$ be a set of shares accepted by VerSS on T. Since R is verifiability-aware and perfect (Definition 5.8), there exists $E = \{\text{sh}_i : i \in C\}$ such that, for any $\text{SH} \subseteq \mathcal{V}(\text{T}, \Sigma) \setminus E$ with $|\text{SH}| = t - f$,

$$\Pr[\text{VerRS}(\text{R}(\text{SH}), \text{T}) = 1] = 1.$$

In TrVer^{R} , after provenance checks via VerSD and VerSS , a dummy set DSH of size $t - f - 1$ is sampled from VK, which lies in $\mathcal{V}(\text{T}, \Sigma) \setminus E$ by the verifiability-awareness. Consider $\text{SH}^* = \text{DSH} \cup \{\text{sh}_{i^*}\}$. Since party i^* is honest, $\text{sh}_{i^*} \in$

$\mathcal{V}(\mathsf{T}, \Sigma) \setminus \mathsf{E}$ except with the probability that some verifiability check fails. Hence $\Pr[\text{VerRS}(\mathsf{R}(\mathsf{SH}^*), \mathsf{T}) = 1] = 1$ unless VerSD , VerSS or VerRS is violated. Therefore, $\Pr[W|W_1] \leq \epsilon_{\text{vss}} + \epsilon_{\text{vsd}} + \epsilon_{\text{vrs}}$, where ϵ_i for $i \in \{\text{vss}, \text{vsd}, \text{vrs}\}$ denotes the maximum success probability of violating the corresponding verifiability property.

Combining the two cases,

$$\begin{aligned} \text{Adv}_{\mathcal{A}, \text{TSS-PV}}^{\text{ni}}(\lambda) &= \Pr[W] = \Pr[W \wedge W_0] + \Pr[W \wedge W_1] \\ &\leq \Pr[W_0] + \Pr[W|W_1] \\ &\leq \epsilon_{\text{psec}} + \epsilon_{\text{vss}} + \epsilon_{\text{vsd}} + \epsilon_{\text{vrs}}, \end{aligned}$$

which is eventually negligible. $\square$